

Table of Contents

1. Introduction	4
1.1. Problem Definition.....	4
1.2. Proposed Solution	4
2. Planning.....	6
2.1. Motivation.....	6
2.2. Initial System Request	6
2.2.1 Business Needs	6
2.2.2 Business Requirements	7
2.2.3 Business Value.....	8
2.2.4 Special Issues or Constraints	8
2.3. Feasibility Analysis.....	8
2.3.1 Technical Feasibility.....	8
2.3.2 Operational Feasibility.....	9
2.3.3 Economic Feasibility	9
2.3.4 Legal & Regulatory Feasibility.....	10
2.4. Initial Project Plan.....	10
3. Analysis.....	12
3.1 Use Case Diagram.....	12
3.2 Use Case Description.....	13
3.3. Activity Diagram	19
3.4. Sequence Diagram	31
4. Design.....	43
4.1. Architecture Design	43
4.1.1. Package Diagram	43
4.2. Class Specification Design	45
4.2.1. Model Package.....	45
4.2.2. View Package.....	56
4.2.3. Control Package	66
4.3. Data Storage Design	73
4.3.1. Overview	73
4.3.2. Detailed Data Storage Design.....	76
4.4. User Interface Design	83
4.4.1. Interface Prototype.....	83

4.4.2. Interface Specification	88
5. Conclusion and Future Work.....	90
5.1. Conclusion	90
5.2. Future Work	91
References.....	92

1. Introduction

1.1. Problem Definition

Homestay management in many regions remains fragmented and manually intensive. Hosts juggle homestay profiles, guest inquiries, bookings, identity verification, check-ins, payments, cleaning schedules, and reviews—often across spreadsheets, messaging apps, and paper records. This manual process doesn't scale across multiple homestay profiles, struggles to maintain consistent service quality, and consumes significant time and human effort. Meanwhile, guests expect seamless, transparent, and trustworthy digital interactions from booking to checkout.

Current practices leave major gaps: guests may not receive timely confirmations or check-in details, ID verification can be inconsistent, cleaning coordination may lack traceability, and payment or refund processes can be confusing. A modern, guest-facing homestay management platform is needed to:

- (i) Present verified booking and stay records (reservations, payments, guest IDs, cleaning logs) clearly.
- (ii) Streamline follow-up actions (ID verification, payments, feedback, issue reporting, scheduling maintenance/cleaning).
- (iii) Build trust through transparent evidence (time-stamped logs, photos, invoices, check-in/out proofs) and consistent communication.

1.2. Proposed Solution

The proposed **Homestay Booking Management System** provides a transparent, automated platform to manage the full guest lifecycle. It integrates homestay profile syndication, secure booking, mandatory guest identity verification, e-payments, and housekeeping workflows into a centralized database while following an event-triggered data-capture policy to ensure fairness, privacy, and trust.

How it works:

1. **Neutral Operation:** The platform runs daily hosting workflows without storing unnecessary personal data beyond what is needed to deliver services.

2. **Trigger on Event:** Only when a business event occurs (booking confirmed, ID verification initiated, payment processed, check-in/out completed, issue reported) does the system collect and store the minimum relevant records (documents, photos, receipts, timestamps).
3. **Guest Identity Verification:** Guests must submit valid government-issued identification to verify their identity before check-in. Accepted documents include national ID card (e.g., Vietnamese CCCD), driver's license, or passport. The platform supports secure capture (scan or photo), liveness/selfie checks where applicable, and automated validation (format, expiry, MRZ/barcode) before completing the booking.
4. **Guest & Stay Records:** Each booking stores consented data (ID validation outcome, payment confirmation, feedback) and links them to auditable records.
5. **Human Review & Notifications:** Low-confidence or sensitive cases (ID mismatch, damage disputes, refund requests, habitability issues) are routed for host review before final actions are taken.
6. **Human Review & Notifications:** Low-confidence or sensitive cases (ID mismatch, screening edge cases, charge disputes, habitability issues) are routed for staff review before official notices or charges are issued to tenants.

Privacy & Security for ID Data:

1. **Consent & Purpose Limitation:** IDs are collected solely to verify guest identity, prevent fraud, and ensure the safety of both guests and hosts.
2. **Data Minimization:** Only fields necessary for verification are processed; non-essential fields (e.g., full ID number) are masked.
3. **Encryption & Access Control:** Documents are encrypted both at rest and in transit; access is restricted by role and logged.
4. **Retention & Deletion:** ID images and verification records are retained only for legally required periods or until checkout completion, then securely deleted.
5. **Auditability:** Every verification or booking action is time-stamped and traceable to ensure compliance and dispute resolution.

2. Planning

2.1. Motivation

Social Motivation: A consistent, transparent homestay management system enhances community trust and hospitality standards. The event-triggered policy respects guest privacy—information is collected only when necessary—and builds credibility by attaching clear evidence (photos, receipts, cleaning logs) to every stay. Verified ID at booking time ensures guest authenticity and safety for hosts.

Technological Motivation: The project integrates modern technologies (document OCR, liveness detection, e-signature, digital payments, RBAC, workflow automation, and structured data models) to deliver a robust real-world solution. Implementing event-triggered capture with identity verification is technically complex yet essential for both efficiency and privacy.

Educational Motivation: The project allows the development team to gain experience in secure identity flows, multi-property architecture, real-time notifications, and guest-centric UX. It promotes ethical automation and privacy-by-design awareness.

Policy & Process Motivation: By capturing personal data only when a legitimate stay event occurs (with explicit guest consent for ID checks), the system demonstrates responsible data handling. Routine communications are logged; evidence for refunds or disputes is encrypted and retained only as required by law; and guests have transparent access to their stay records and issue-resolution channels.

2.2. Initial System Request

2.2.1 Business Needs

On one hand, property owners face limited exposure to potential tenants, inefficient communication, and income loss from vacant properties. On the other hand, tenants struggle with fragmented homestay profiles, a lack of transparent information, and time-consuming search processes. However, intermediary organizations often operate across multiple platforms, leading to inefficiencies and lower user satisfaction. The proposed platform addresses these pain points by centralizing property listings, improving transparency, and enabling real-time

updates. This creates a more accessible, efficient, and sustainable housing ecosystem for all stakeholders.

2.2.2 Business Requirements

The platform must meet the following key requirements to ensure functionality, user satisfaction, and scalability:

a) Accessibility and User Accounts

- Tenants can search for potential housing services without an account.
- Users can sign up or log in using either email or phone number for flexible access.

b) Property Listing and Management

- Homeowners can list properties with details such as area, price, furniture, and real images.
- The homestay profile will automatically update or be hidden once a property is rented or sold.
- Real-time availability ensures accurate and up-to-date information.

c) Search and Filtering

- Users can filter properties by type, price, area, distance, and reviews.
- Users can search based on their specific needs, such as the number of occupants, desired dates, and whether children are included.
- Integrated maps and contact details enable direct communication with homeowners.
- A property comparison feature allows users to view multiple homestays side by side.

d) Community and Interaction

- Optional chat function for sharing local tips, events, and services.
- Tenants can leave reviews after renting, improving transparency and trust.

e) Performance and Usability

- The web application will be optimized for all major browsers.
- The interface must be intuitive for both tenants and property owners, minimizing the learning curve and improving engagement.

2.2.3 Business Value

- Additional income streams through advertising, transaction fees, and premium features gives revenue generation.
- The centralized platform reduces marketing and advertising expenses while streamlining the rental process for both owners and tenants.
- Faster transactions and improved communication significantly increase efficiency and reduce time spent on administrative tasks.
- The platform enables the collection and analysis of data on tenant preferences, market trends, and property performance to support strategic decision-making.

2.2.4 Special Issues or Constraints

- The platform must integrate seamlessly with existing property databases and ensure real-time updates.
- The platform should maintain high security standards to protect personal and transaction data.
- A stable infrastructure is required to handle a potentially large number of concurrent users.
- Legal compliance with data protection and housing regulations must be maintained at all times.

2.3. Feasibility Analysis

2.3.1 Technical Feasibility

- **Current Technology Availability:** The project can leverage existing and mature technologies in image recognition, data processing, and cloud infrastructure:
 - **Frontend:** Developed using HTML, CSS, JavaScript, and React for an interactive and responsive interface.
 - **Backend:** Implemented with Node.js and Express (or Java Spring Boot) to handle user authentication, bookings, and payment APIs.
 - **Database:** Relational database for structured data management

- **Cloud Integration:** Cloud-based platforms such as AWS, Azure, or Google Cloud can be utilized to host the system, ensuring scalability and high availability.
- **Developer Expertise:**
 - **Familiarity with the application:** Team members have moderate expertise in frontend development and API integration.
 - **Familiarity with the Technology Stack:** The team has prior experience in SQL and backend development.
- **Project Size:** The project is considered medium-scale, requiring a team of 4-6 members including backend and frontend developers, database administrators, and testers. A 10–12 week development timeline is realistic for completing system design, backend integration, web interface development, testing, and deployment.
- **Compatibility:** The web-based control and monitoring interface will be optimized for major browsers such as Google Chrome, Mozilla Firefox, and Safari.

2.3.2 Operational Feasibility

- **User Experience:** Intuitive interface for both hosts and guests to post, browse, and book homestays easily.
- **Host Management:** Simple verification process to ensure legitimacy while avoiding complex barriers to entry.
- **Tenant Support:** Search filters, booking history, and customer service chat ensure a smooth rental experience.
- **Maintenance:** Regular updates and monitoring to ensure system reliability and responsiveness.

2.3.3 Economic Feasibility

- **Cost of Development:**
 - **Initial Costs:** Mainly developer salaries, hosting services, and domain registration.

- **Operational Costs:** Server maintenance, database management, and marketing.
- **ROI Expectation:** High potential returns through increased transactions and repeat users due to platform reliability and trust.
- **Revenue Generation and Cost Efficiency:**
 - **Revenue Sources:** Service fees, premium listing options, and targeted advertising.

2.3.4 Legal & Regulatory Feasibility

- **Compliance:** Adheres to local housing and data protection laws (e.g., GDPR compliance for user data).
- **Verification:** Hosts must verify property ownership and identity to prevent fraud.
- **Terms & Policies:** Clear refund, cancellation, and liability policies to protect both hosts and guests.
- **Licensing:** Ensures homestay operations meet regional short-term rental regulations.

2.4. Initial Project Plan

The project will adopt an **Agile development methodology**, enabling iterative progress, adaptability to changing market needs, and continuous feedback from hosts and guests. This approach suits the dynamic nature of the homestay rental business, which involves multiple stakeholders (property owners, guests, and platform administrators) and requires incremental delivery of features for smooth onboarding and usability.

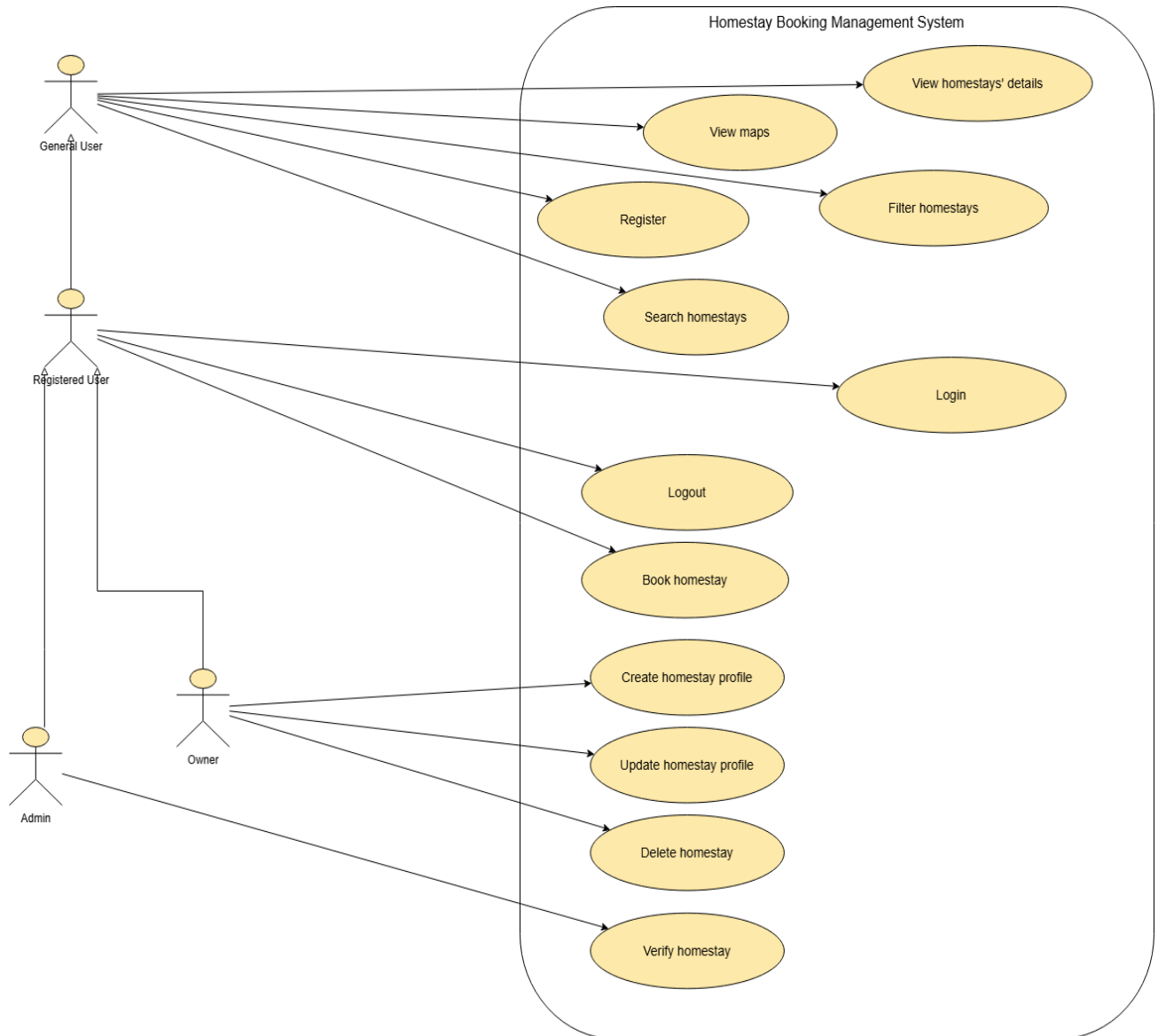
Key reasons for choosing the Agile methodology:

- **Flexibility:** Pricing models, booking policies, and integration with payment gateways may evolve; Agile allows the system to adapt quickly without major rework.
- **Early Value Delivery:** Core modules such as Property Listing, Booking Management, and Payment Processing can be released early for testing and feedback.

- **Continuous Feedback:** Hosts and guests can provide input during each sprint, ensuring the platform meets usability, security, and convenience standards.
- **Risk Mitigation:** Frequent sprint reviews help identify potential issues early, reducing risks related to data accuracy, payment failures, and user experience.
- **User Focus:** Each iteration prioritizes improving transparency, ease of booking, and host-guest communication to enhance overall satisfaction.

3. Analysis

3.1 Use Case Diagram



3.2 Use Case Description

Id	UC 1.1
Name	View homestay's details
Short description	View the details regarding address, price, and available services of a specific homestay
Precondition	1. Users need to access the website. 2. All data about the homestay has been verified and listed to the dashboard.
Postcondition	All data about the homestay appears on the user's screen.
Error situation	1. Internet Interruption → Display network error message. 2. System under maintenance → Display maintenance notice. 3. Homestay removed during viewing → Show "Homestay unavailable"
System state in the event of an error	Users cannot view the homestay's details.
Actor	General User (Unregistered/Registered)
Trigger	Users click on the thumbnail of a homestay
Standard process	1. Some thumbnails (pictures, address and price) of the homestay are displayed on the screen. 2. Users surf and click on their favourite homestay. 3. The system leads Users to a new screen of all information of the homestay.
Alternative processes	None

Id	UC 1.2
Name	View maps
Short description	User views address on Google Maps.
Precondition	1. Users clicked to view homestay's details. 2. The Directions button should be displayed next to the address information.
Postcondition	The system switches the tab to Google Maps to display the address to be searched.
Error situation	1. Internet Interruption → Display connection error 2. Google Maps cannot find information about the homestay address → Show "Location not found."
System state in the event of an error	Users cannot see addresses on Google Maps.
Actor	General User
Trigger	Users want to find the address on Google Maps
Standard process	1. User clicks on the address 2. The system will switches tab to Google Maps
Alternative processes	None

Id	UC 1.3
Name	Filter homestays
Short description	Users filter homestays by address, availability, price or services.
Precondition	The user is on the dashboard site.
Postcondition	Users receive a list of satisfied homestays/rooms or the screen displays "Not found".
Error situation	No Internet connection → Display connection error No matches found → System displays "No homestays found matching your criteria."
System state in the event of an error	Users cannot view the filtered results.
Actor	General User
Trigger	Users want to find homestays/rooms with a specific condition.
Standard process	1. Users click on the Filter button and select criteria (Price Range, Location, Room Count, Amenities). 2. The system returns posts of satisfied homestays/rooms.
Alternative processes	None

Id	UC 1.4
Name	Search homestay
Short description	User search for specific homestay by name
Precondition	The user is on the dashboard.
Postcondition	The screen display the homestays with the matching name
Error situation	No Internet connection → Display connection error
System state in the event of an error	No search results displayed.
Actor	General Users
Trigger	Users want to search for specific homestay
Standard process	1. On the dashboard, the user enters the name of the homestay 2. The system returns the homestays with the same name
Alternative processes	None

Id	UC 1.5
Name	Register
Short description	Users create their account to use other functions.
Precondition	Users access the website.
Postcondition	Users have their own account to log in and use other functions
Error situation	Email already exists → System displays "Account already exists."

System state in the event of an error	Account not created.
Actor	General User
Trigger	Users want to create a new account on the boarding homestay website.
Standard process	1. User enters email and password on the form. 2. User clicks the Register button. 3. The system verifies the form with available username. 4. The system creates a new account with the given username and password. 5. The system displays “Successfully register!”
Alternative processes	None

Id	UC 2.1
Name	Login
Short description	Registered Users log in to their account to use other functions.
Precondition	Registered User needs to access the website and has created an account.
Postcondition	Registered Users can use other functions.
Error situation	The Registered User's name or password is incorrect.
System state in the event of an error	Registered Users cannot login to the website.
Actor	Registered User
Trigger	Registered User wants to log in to the boarding homestay website.
Standard process	1. Registered User enters the correct email and password. 2. Registered User clicks the Login button. 3. The system verifies the account. 4. The system displays the view for owners.
Alternative processes	1. If the username or password is incorrect, the owner re-enters their account info. 2. If the owner hasn't had an account, the system notifies the owner to register a new account with necessary info.

Id	UC 2.2
Name	Logout
Short description	Registered Users log out of their account after finishing the session.
Precondition	Registered User has logged in successfully.
Postcondition	Registered Users can view the General Dashboard only.
Error situation	No Internet connection
System state in the event of an error	Session remains active.
Actor	Registered User

Trigger	Registered User wants to log out of the boarding homestay website.
Standard process	1. Registered User clicks the Log out button. 2. The system displays the General Dashboard view.
Alternative processes	None

Id	UC 2.3
Name	Book homestay
Short description	Registered Users book a homestay of their choice.
Precondition	Registered User has logged in successfully.
Postcondition	Registered Users successfully book the homestay.
Error situation	1. No Internet connection 2. The homestay became unavailable when user was booking.
System state in the event of an error	Registered Users cannot book the homestay.
Actor	Registered User
Trigger	Registered User wants to book a homestay
Standard process	1. Registered User clicks the Book homestay Button 2. Registered User fill in the time range to book the homestay 3. The system notifies the user “Book successfully. Waiting for Owner’s response”
Alternative processes	None

Id	UC 3.1
Name	Create Homestay Profile
Short description	Owners fill out the Create form completely and wait for Admin confirmation.
Precondition	Owners had been logged into the system.
Postcondition	The system notifies the owner to wait for the homestay approval.
Error situation	1. Form is not accepted. 2. Form could not be submitted due to missing info.
System state in the event of an error	Owners cannot submit the form.
Actor	Owner
Trigger	Owners want to create a homestay

Standard process	1. Click "Add New Homestay" button 2. The Create form will appear. 3. Host fills in details (Address, Pricing, Photos, Description). 4. Owner click to the "Submit" button 5. System changes status to "Pending Approval".
Alternative processes	None

Id	UC 3.2
Name	Update Homestay Profile
Short description	Owners fill the Update form.
Precondition	Homestay Profile in the latest archive state.
Postcondition	Homestay Profile in a new updated state.
Error situation	Form could not be submitted due to missing info.
System state in the event of an error	There are no updates to this homestay profile.
Actor	Owner
Trigger	Owners want to update the homestay profile.
Standard process	1. The owner click "Update" button 2. The Update form will appear (the same as the Create form) with latest data 3. The owner fills updated data to the form. 4. The owner clicks the "Submit" button. 5. System saves changes. Note: Critical changes (Address/Price) will trigger re-verification from admin
Alternative processes	None

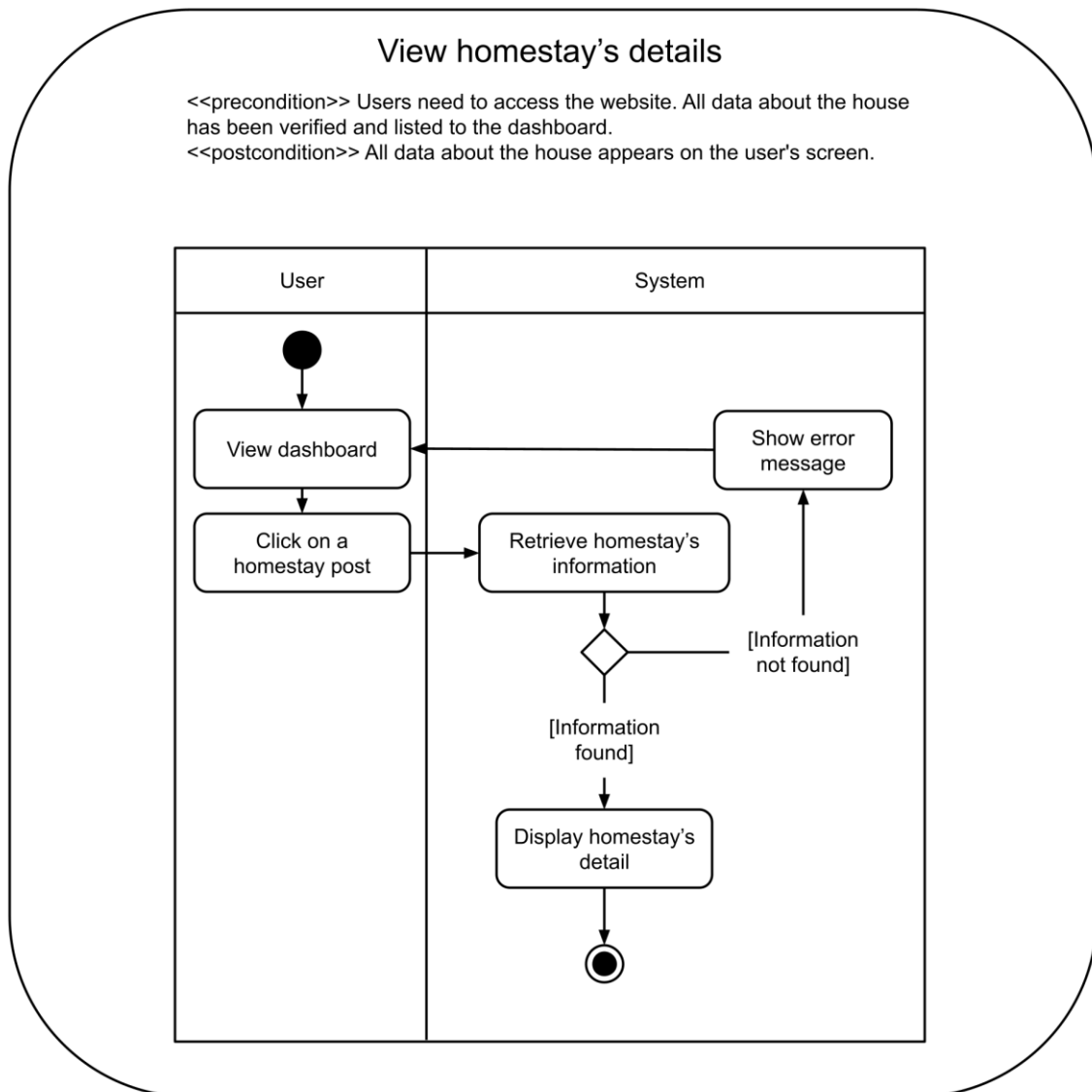
Id	UC 3.3
Name	Delete Homestay
Short description	Owners delete their homestay profile
Precondition	The homestay profile is still on the dashboard.
Postcondition	The homestay profile is no longer on the dashboard. Status in the DB is set to “Archived” and remains for historical reporting.
Error situation	The system cannot receive the command.
System state in the event of an error	The homestay profile cannot be deleted.
Actor	Owner
Trigger	Owner wants to delete the post
Standard process	1. Click the "Delete" button on the post. 2. The system displays "Do you want to delete this homestay profile? Yes or No" 3. The owner chooses Yes.

	4. The data will be removed from the dashboard and update status to “Archived” in DB.
Alternative processes	1. If the owner chooses No, the homestay profile will not be deleted.

Id	UC 4.1
Name	Verify Homestay
Short description	Admin checks information in the owner's Homestay Details and clicks "Approve" or "Reject".
Precondition	Homestay Profile are filled with adequate information
Postcondition	Admin approves or rejects the homestay.
Error situation	No internet connection
System state in the event of an error	Admin cannot verify the homestay.
Actor	Admin
Trigger	Admin receives the list of pending Homestay for approval.
Standard process	1. Admin reads and checks the provided information. 2. If the information is qualified, the admin clicks "Approve". Otherwise, "Reject". 3. If Approved: The system saves the information to the database. 3.1. The homestay and all its information are visible on the dashboard, if approved. 4. The system notifies admin "You have approved/rejected a homestay" and Host of the approval/rejection reasons.
Alternative processes	None

3.3. Activity Diagram

UC 1.1: View Homestay's Details

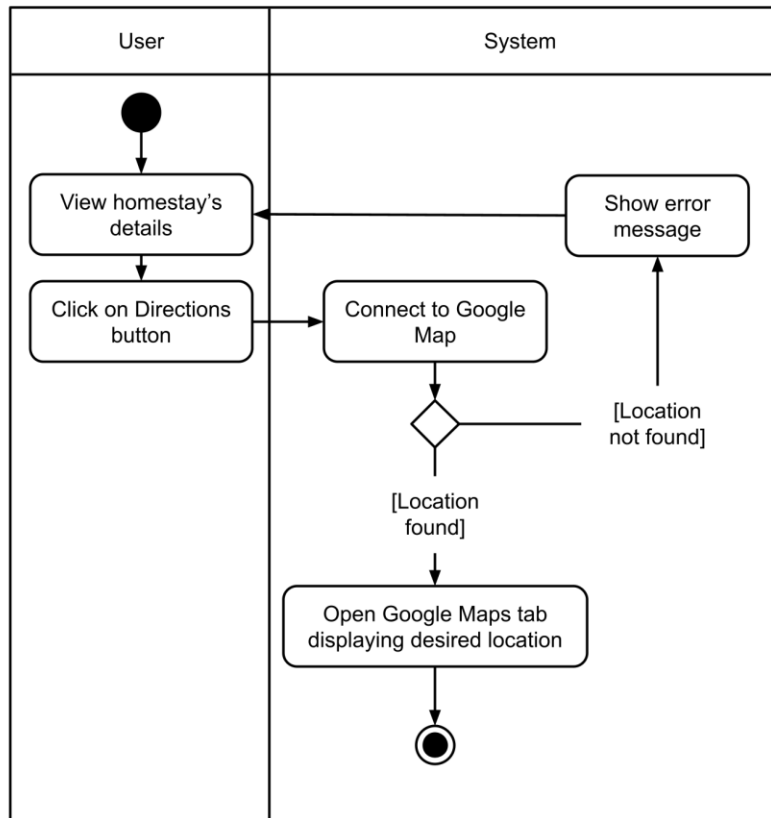


UC 1.2: View Maps

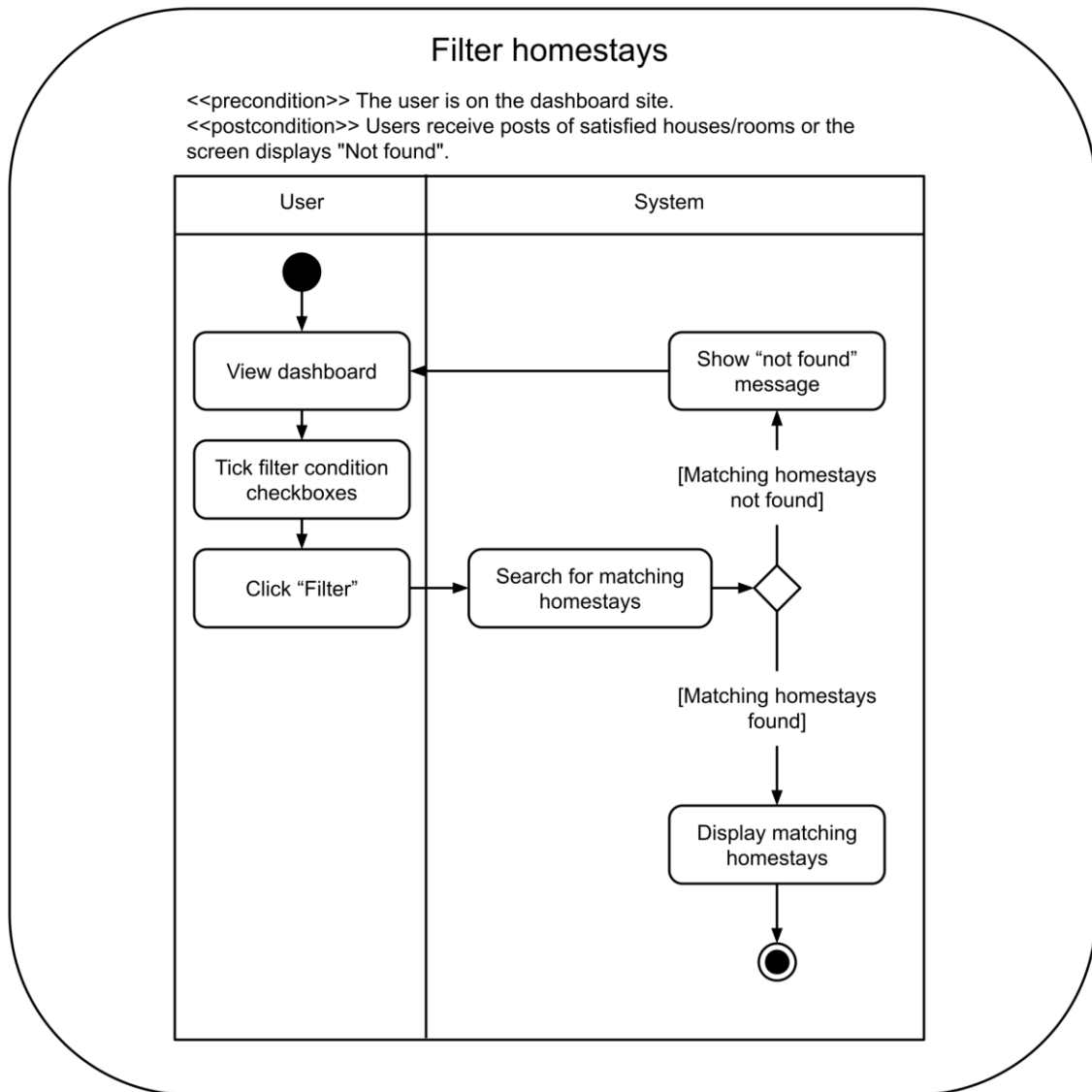
View maps

<<precondition>> Users clicked to view homestay details and see available information. The Directions button should be displayed next to the address information.

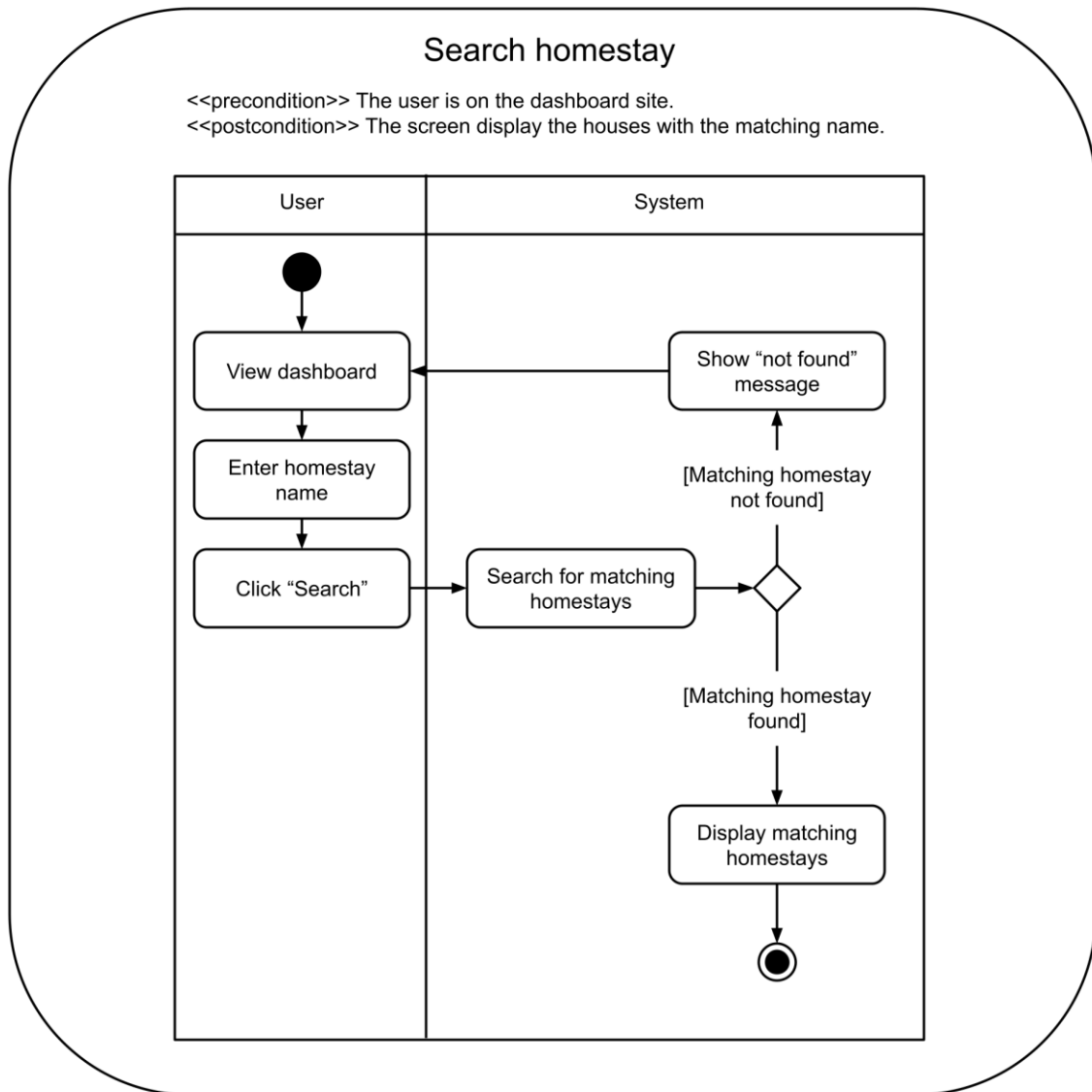
<<postcondition>> The system switches the tab to Google maps to display the address to be searched..



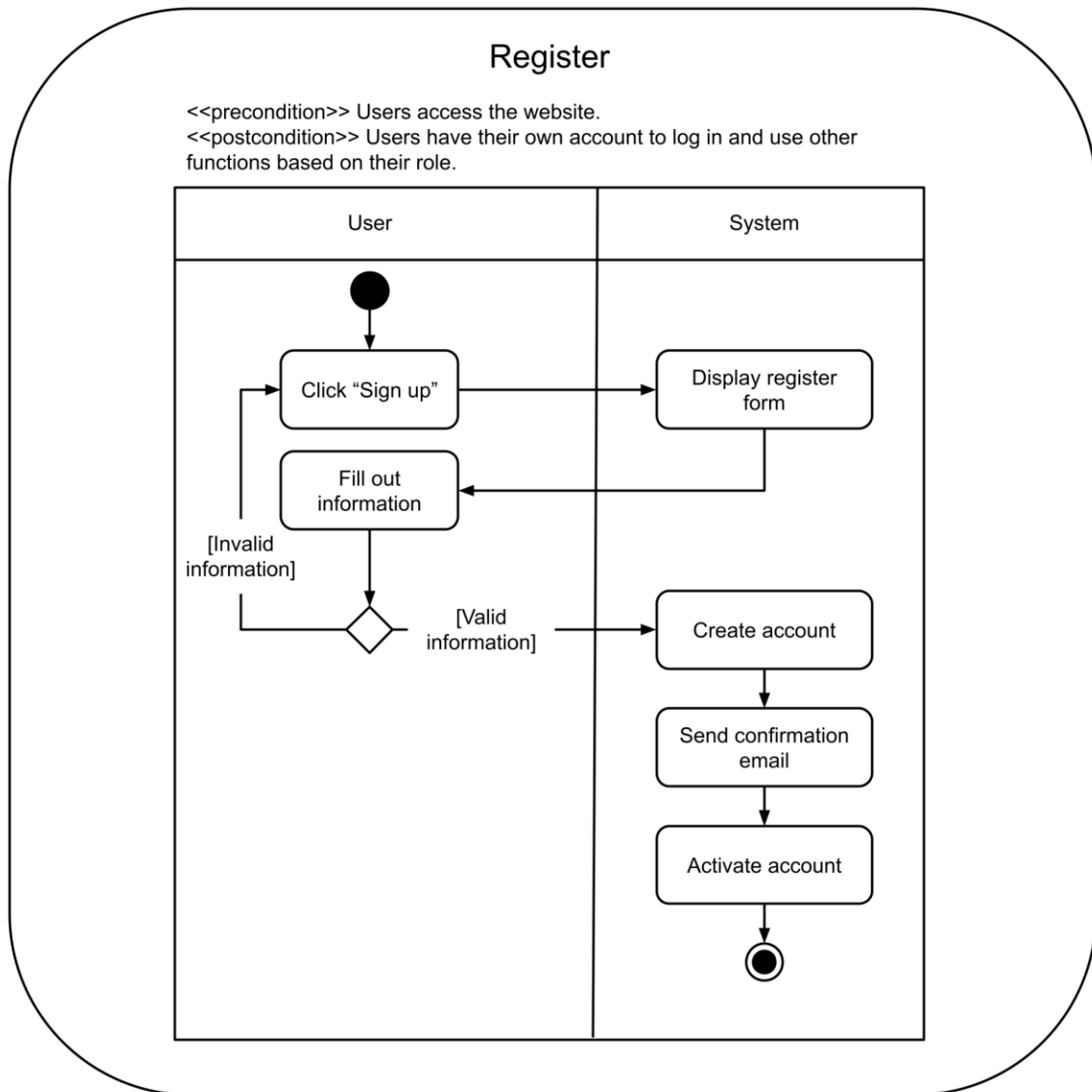
UC 1.3: Filter Homestays



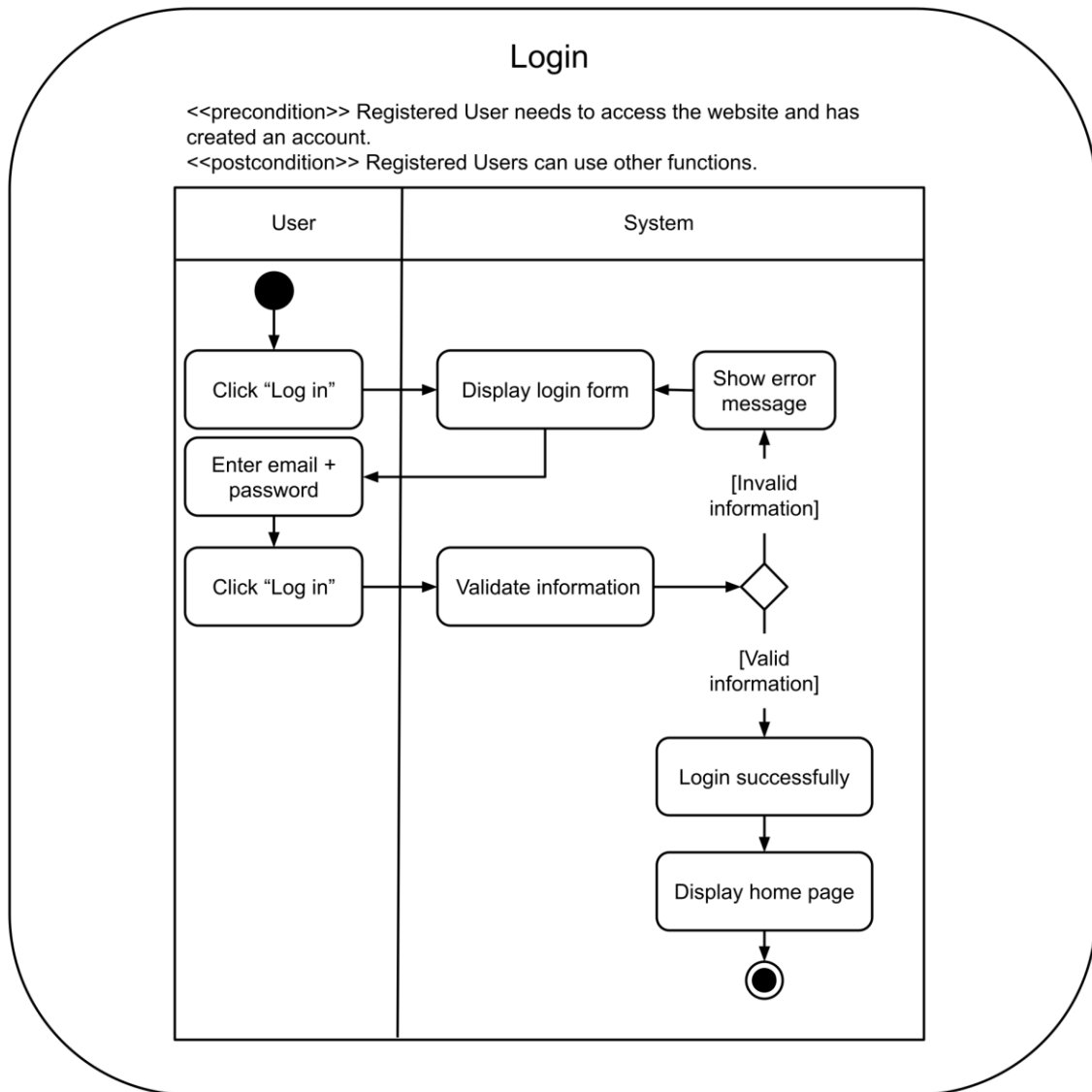
UC 1.4: Search Homestay



UC 1.5: Register

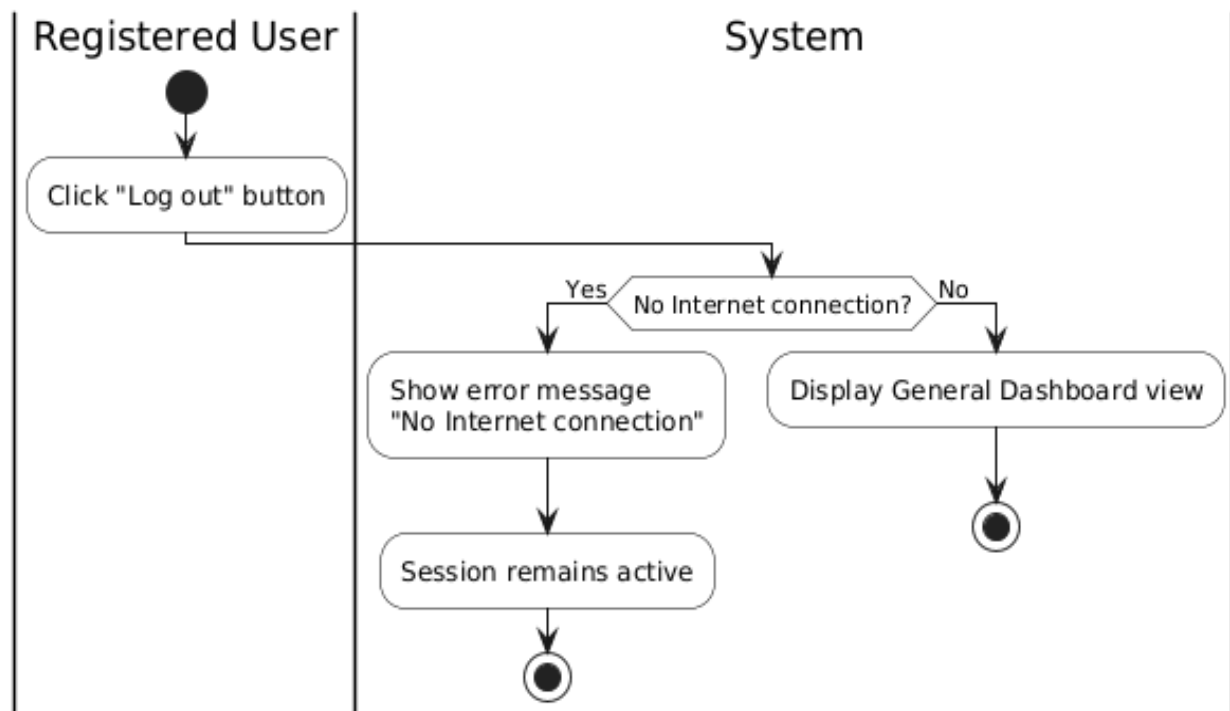


UC 2.1: Login

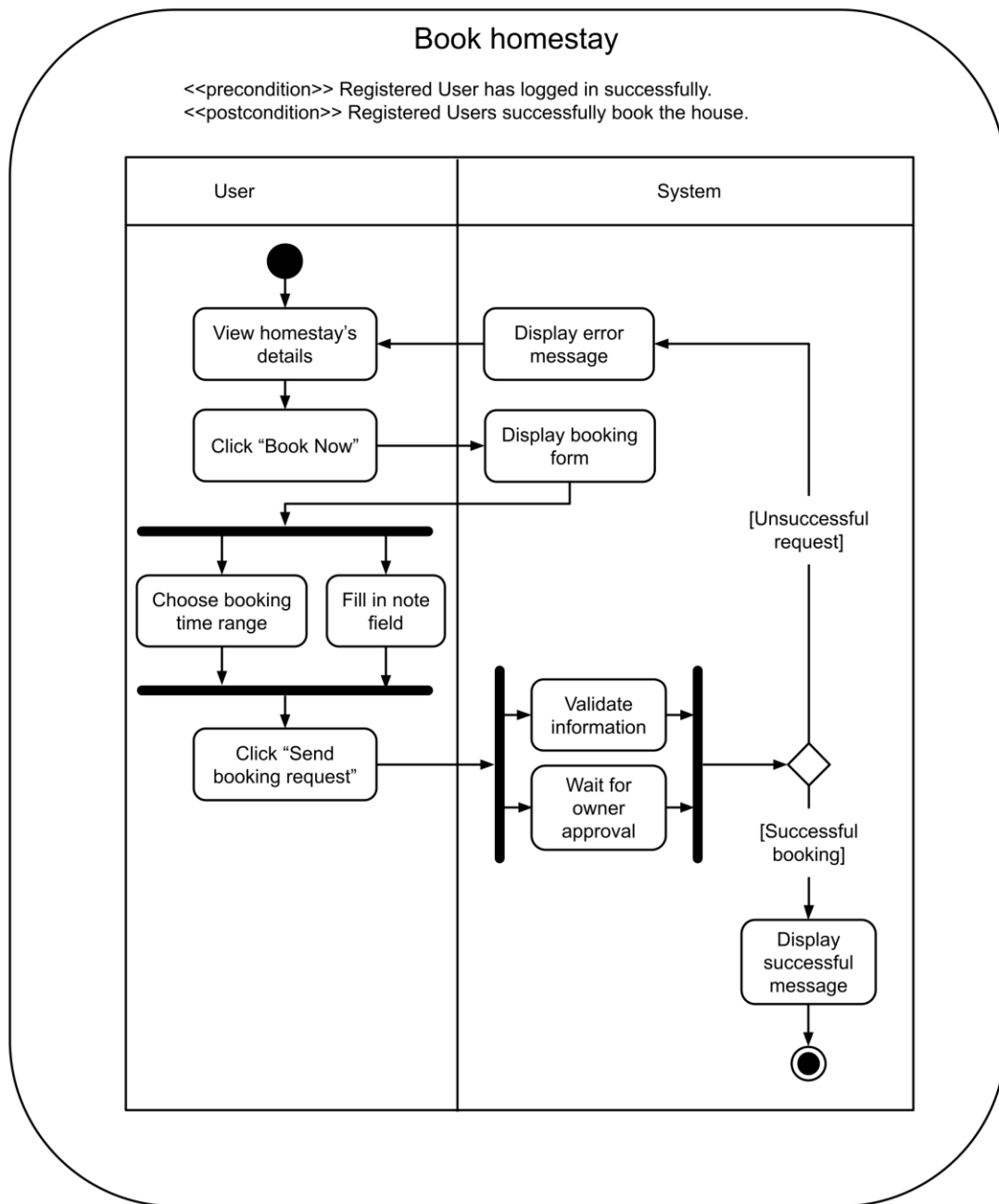


UC 2.2: Logout

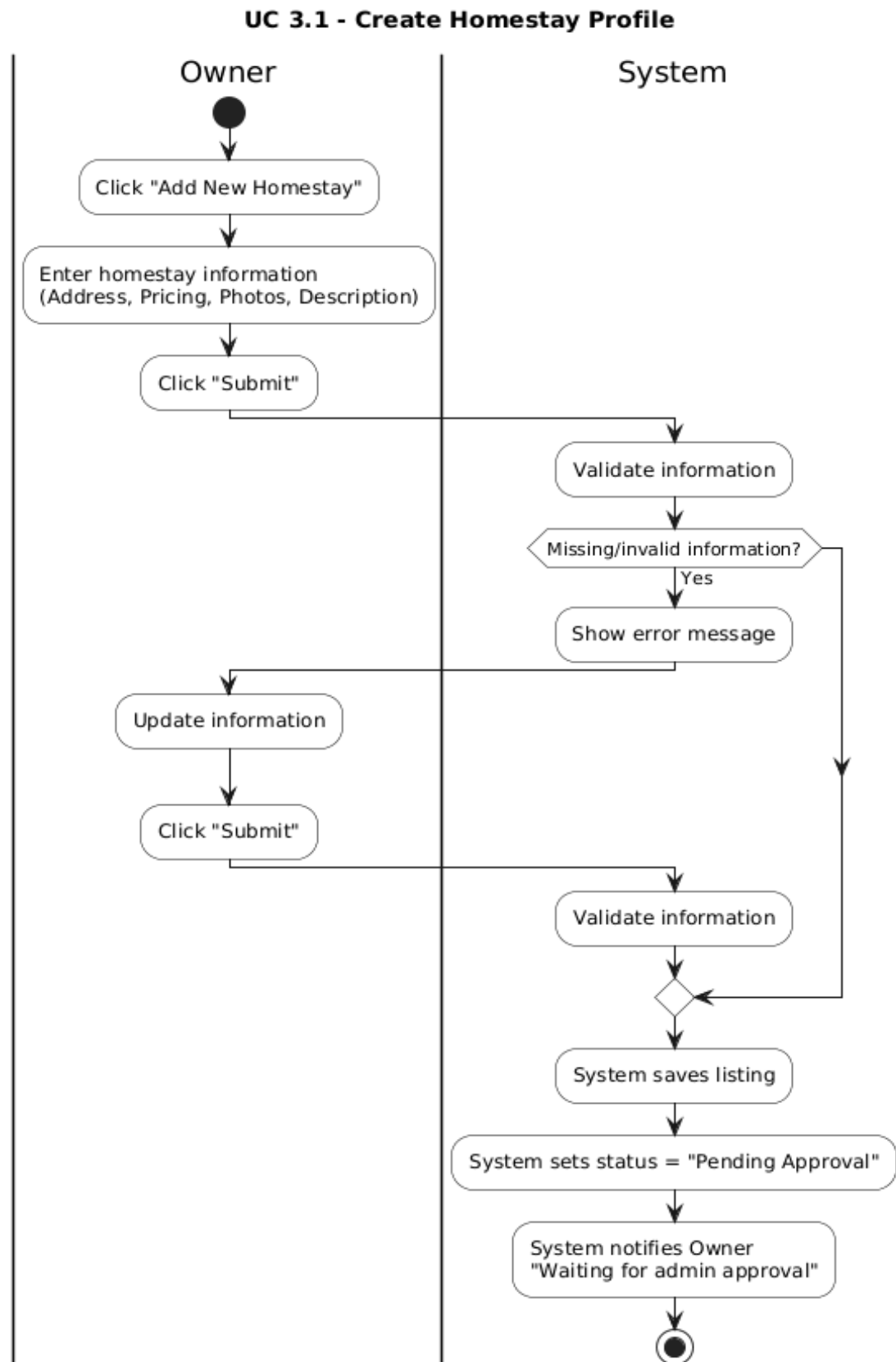
UC 2.2 - Logout (Activity Diagram)



UC 2.3: Book Homestay

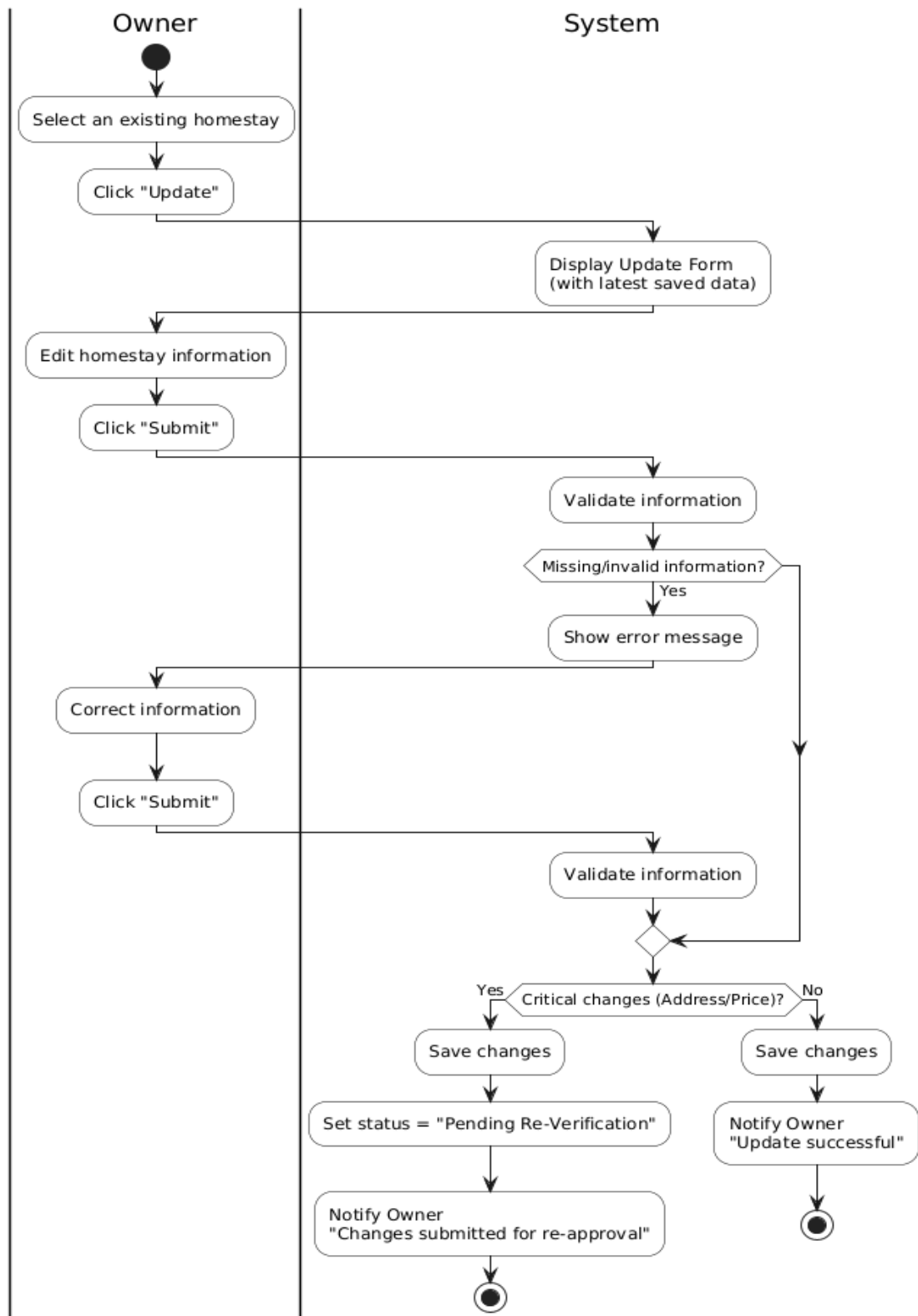


UC 3.1: Create Homestay Profile

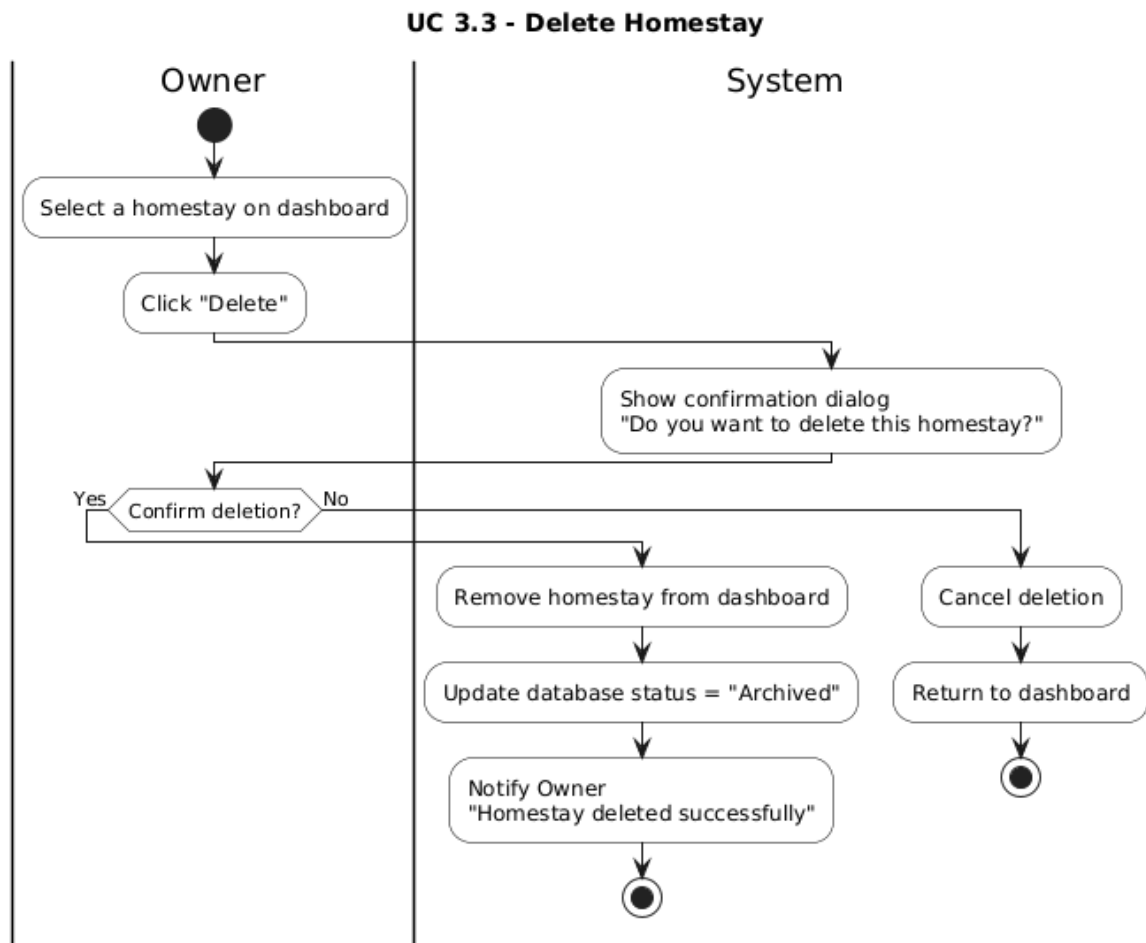


UC 3.2: Update Homestay Profile

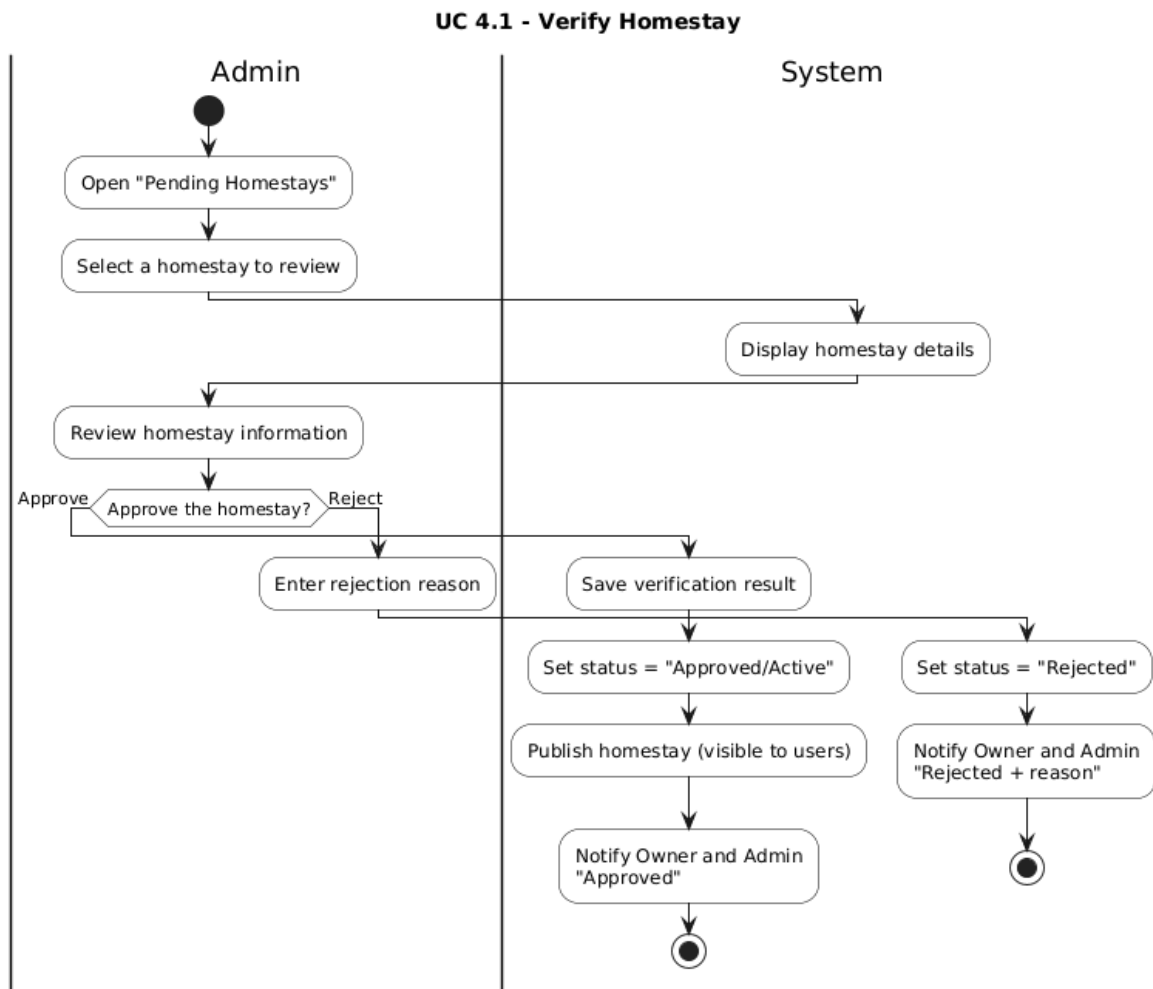
UC 3.2 - Update Homestay Profile



UC 3.3: Delete Homestay

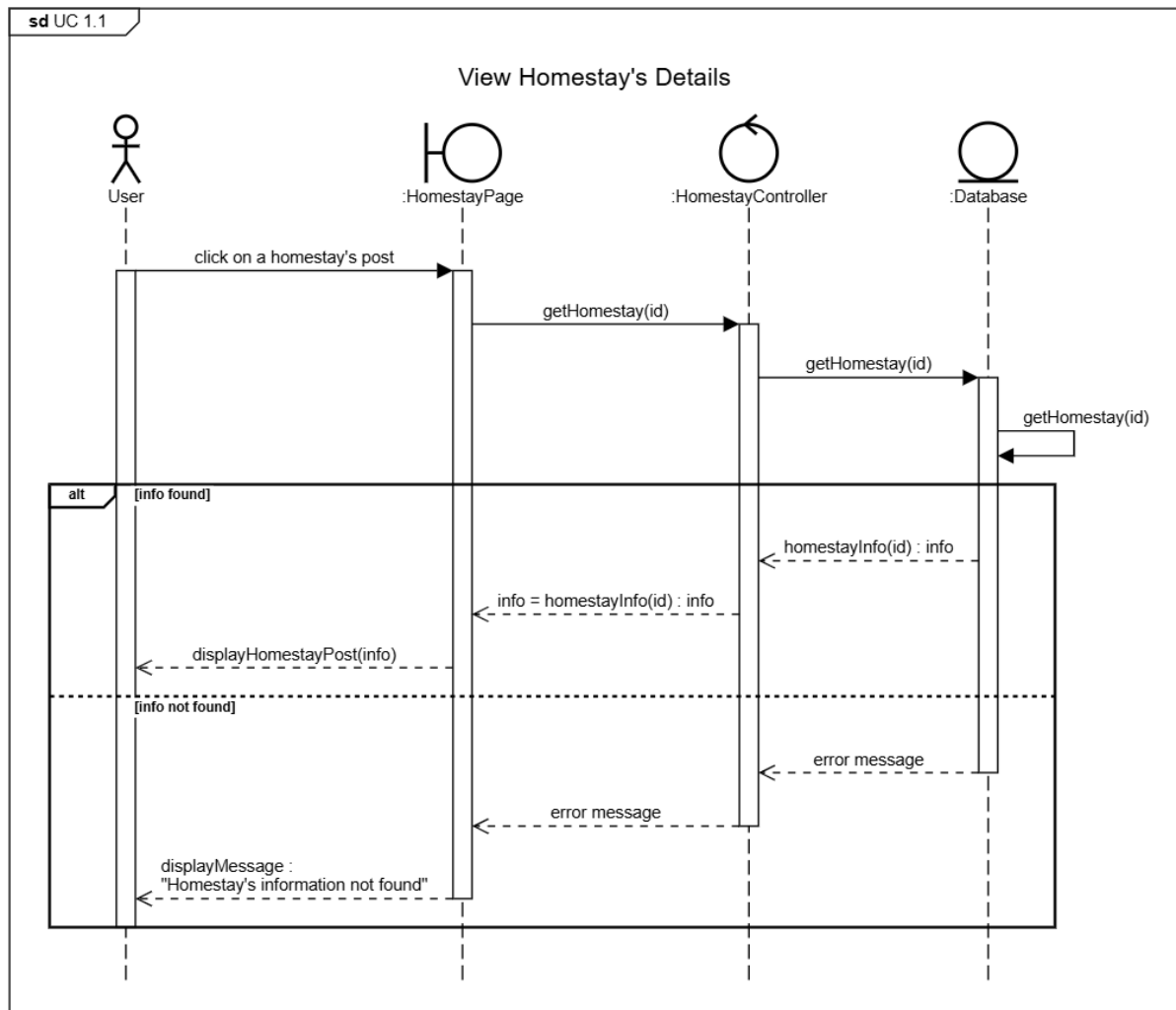


UC 4.1: Verify Homestay

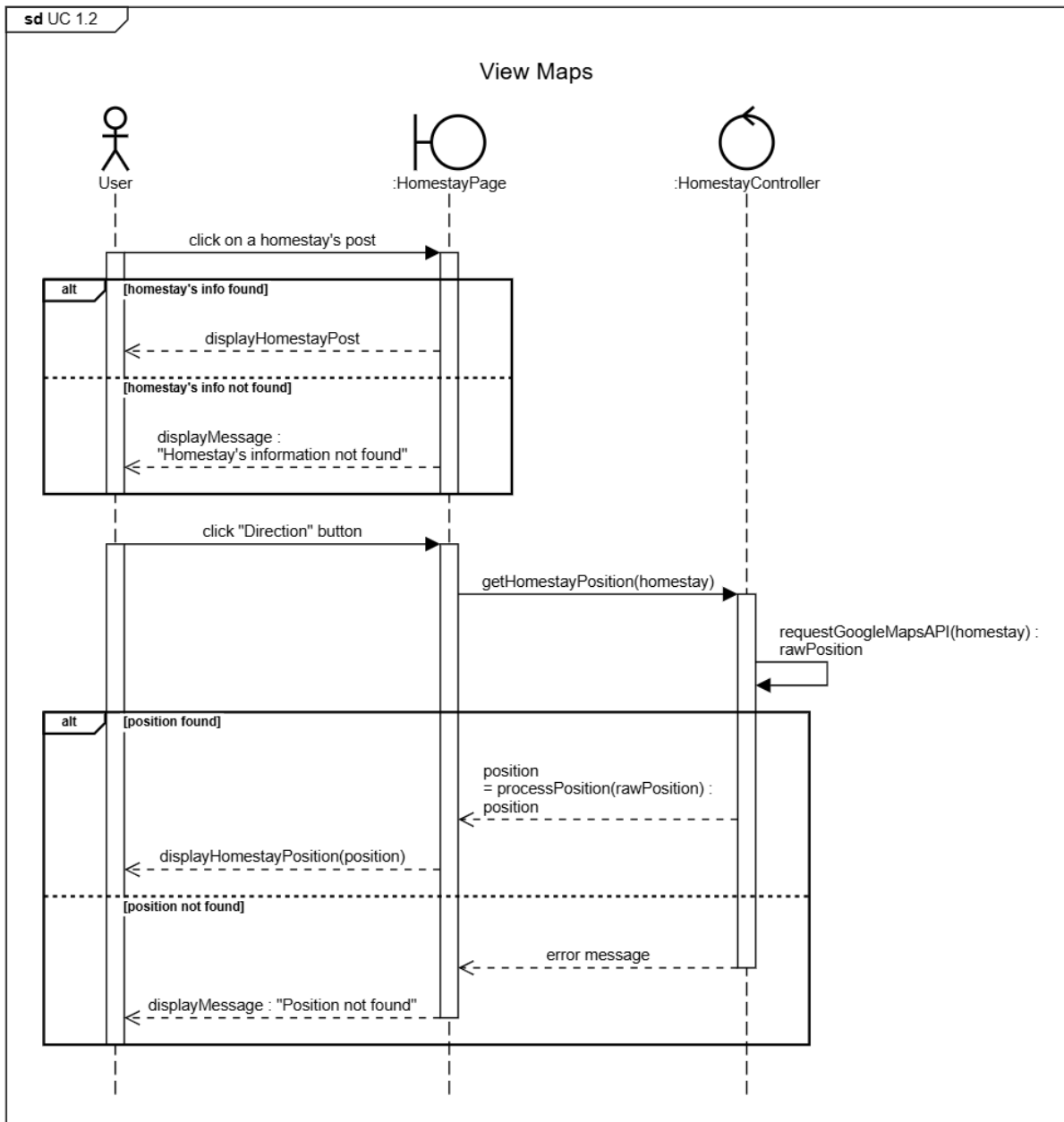


3.4. Sequence Diagram

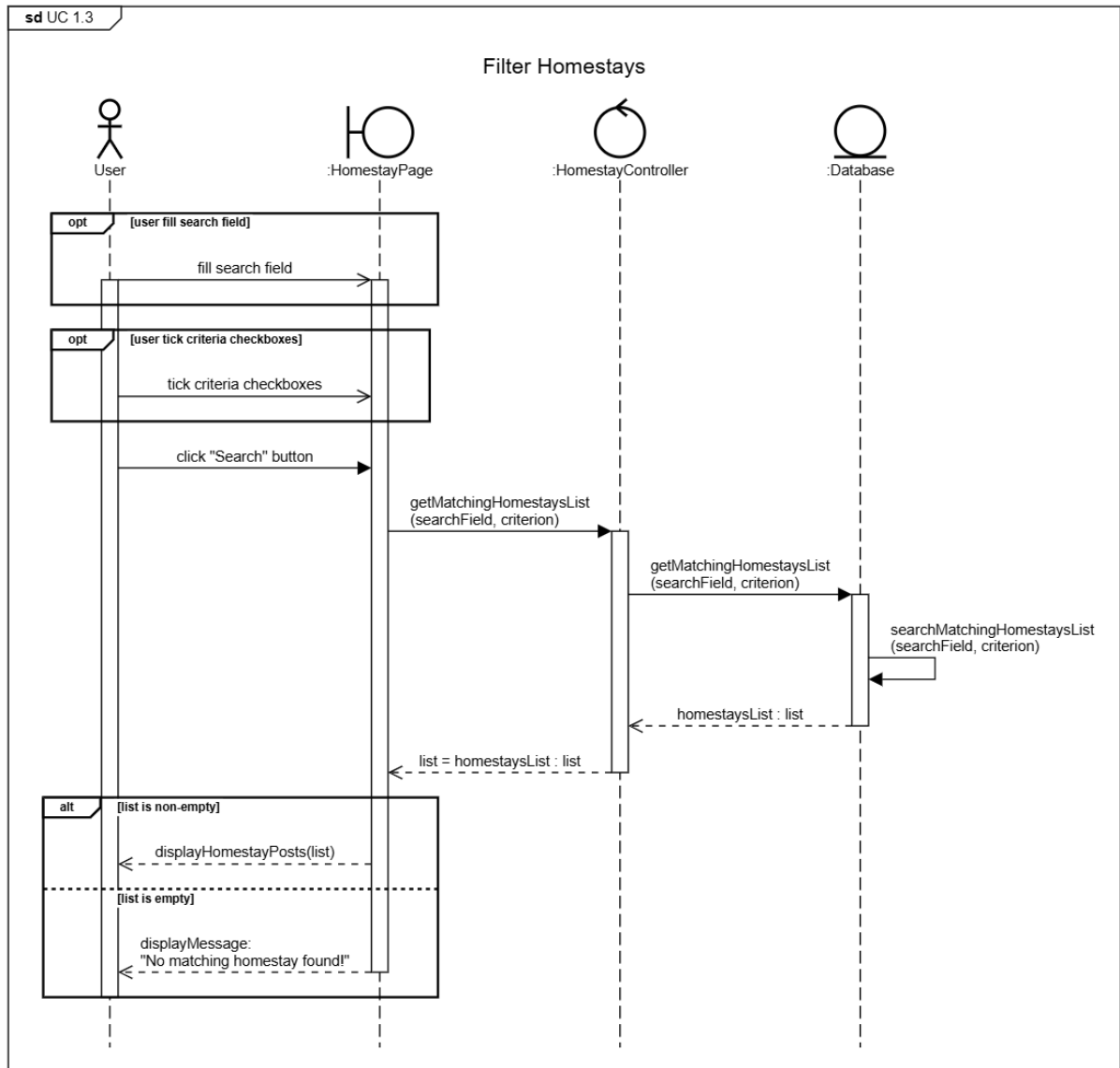
UC 1.1: View Homestay's Details



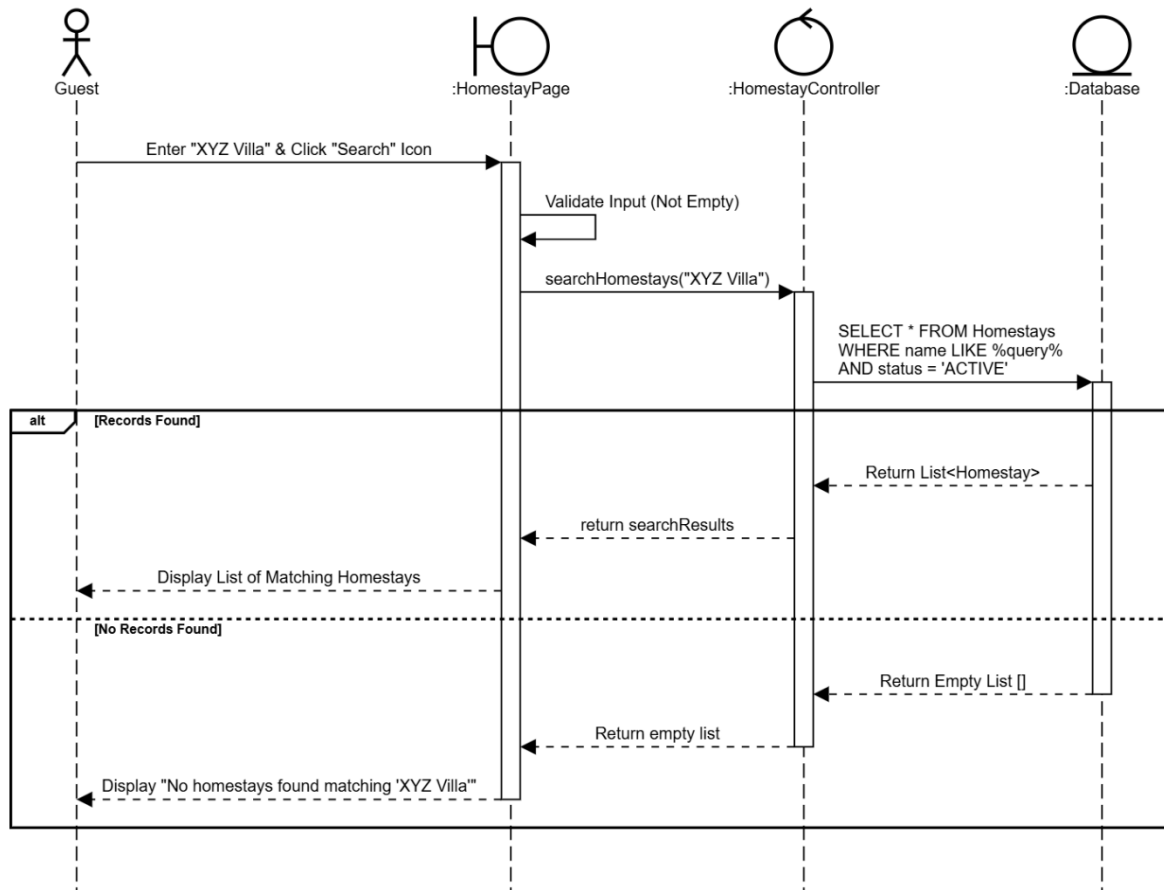
UC 1.2: View Maps



UC 1.3: Filter Homestays

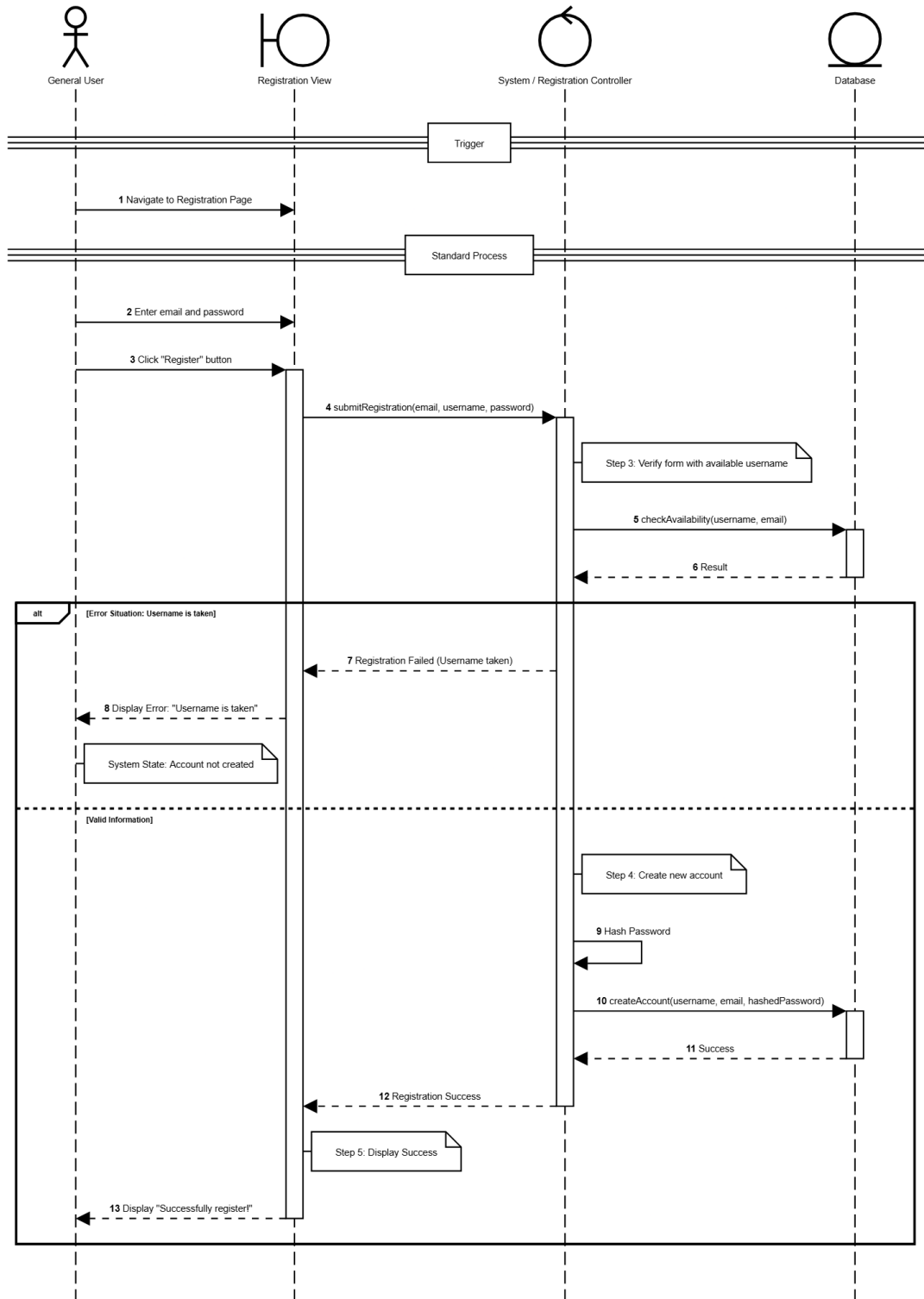


UC 1.4: Search Homestay



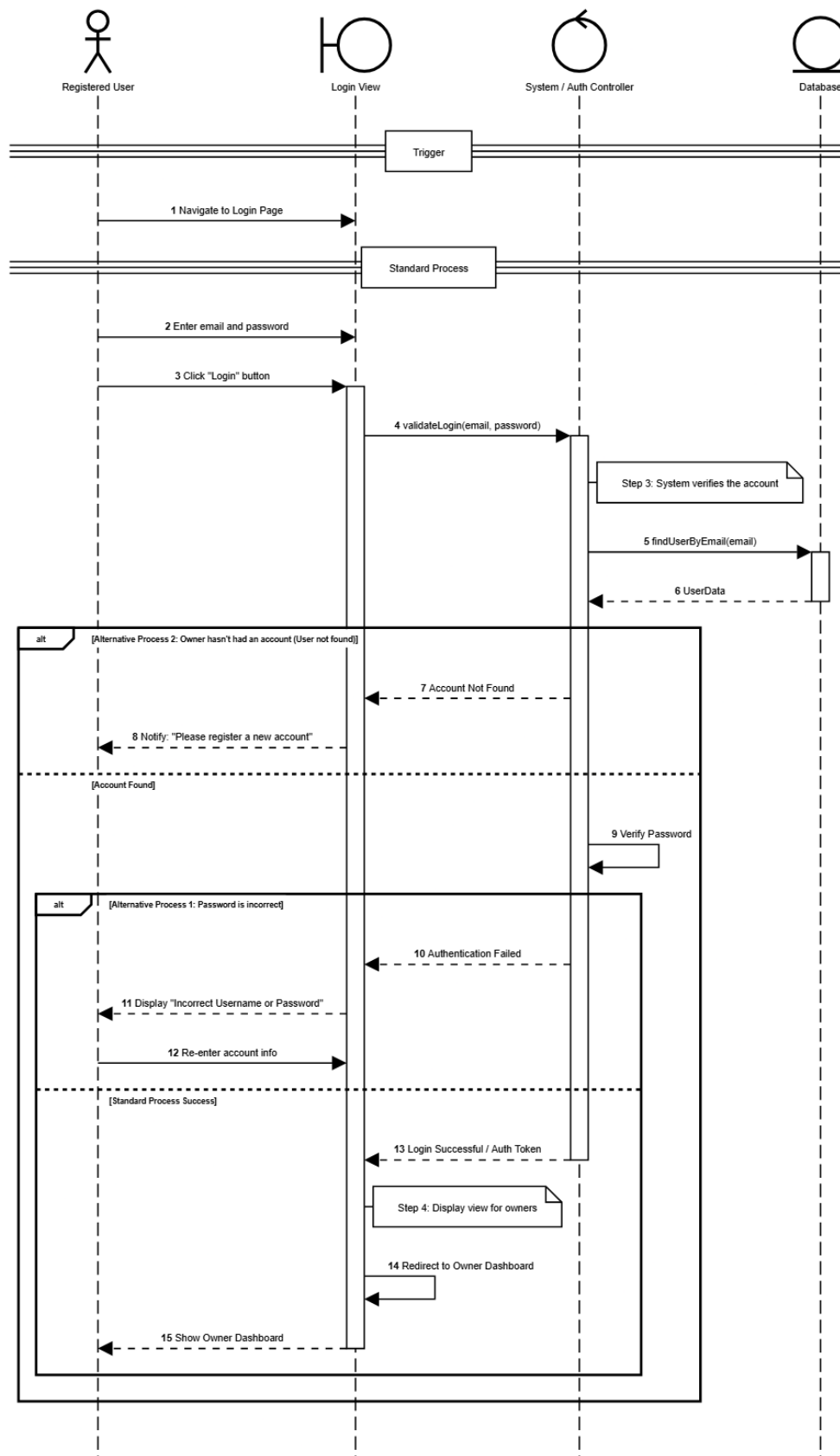
UC 1.5: Register Account

UC 1.5 - Register Sequence Diagram

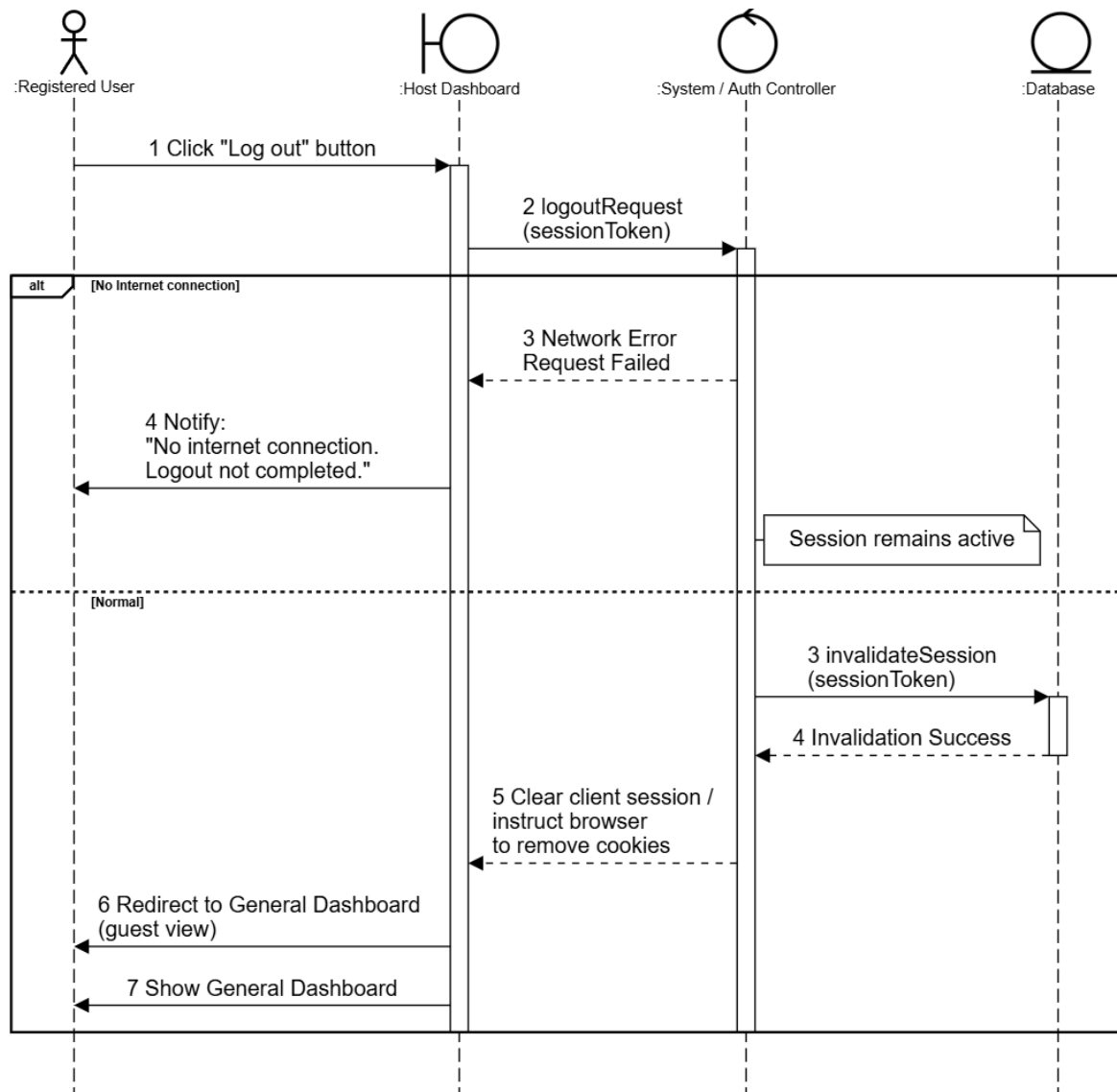


UC 2.1: Login

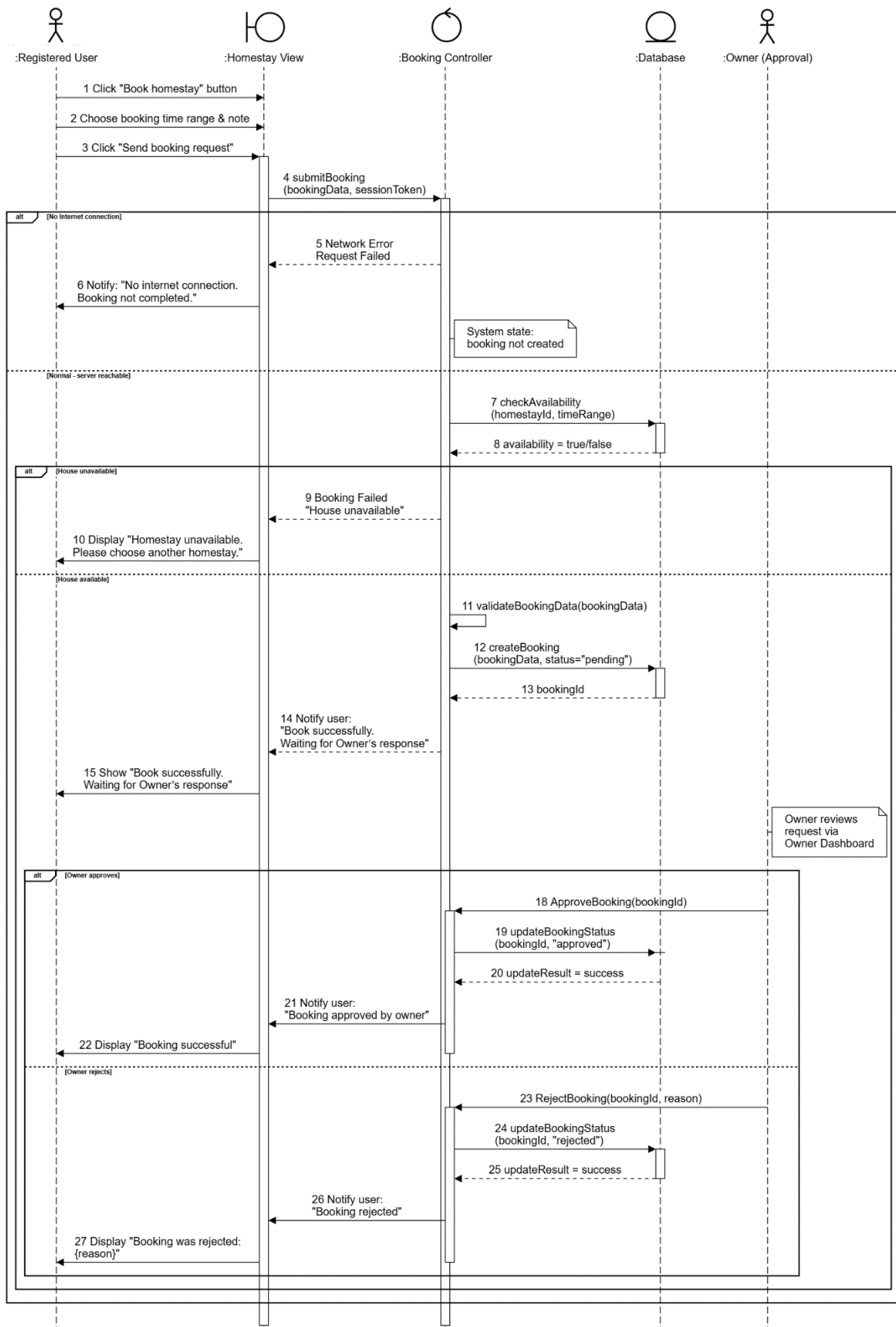
UC 2.1 - Login Sequence Diagram



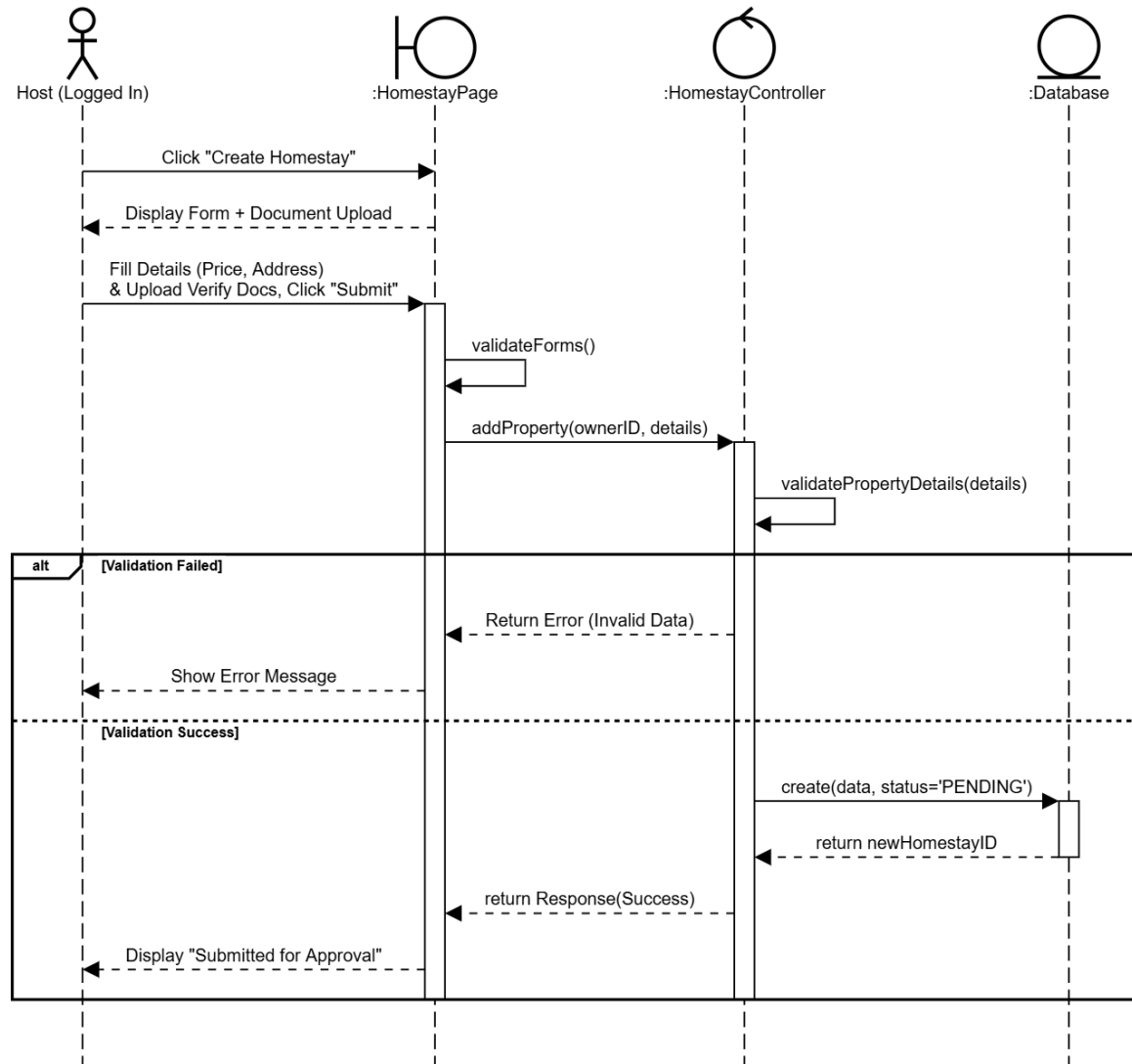
UC 2.2: Logout



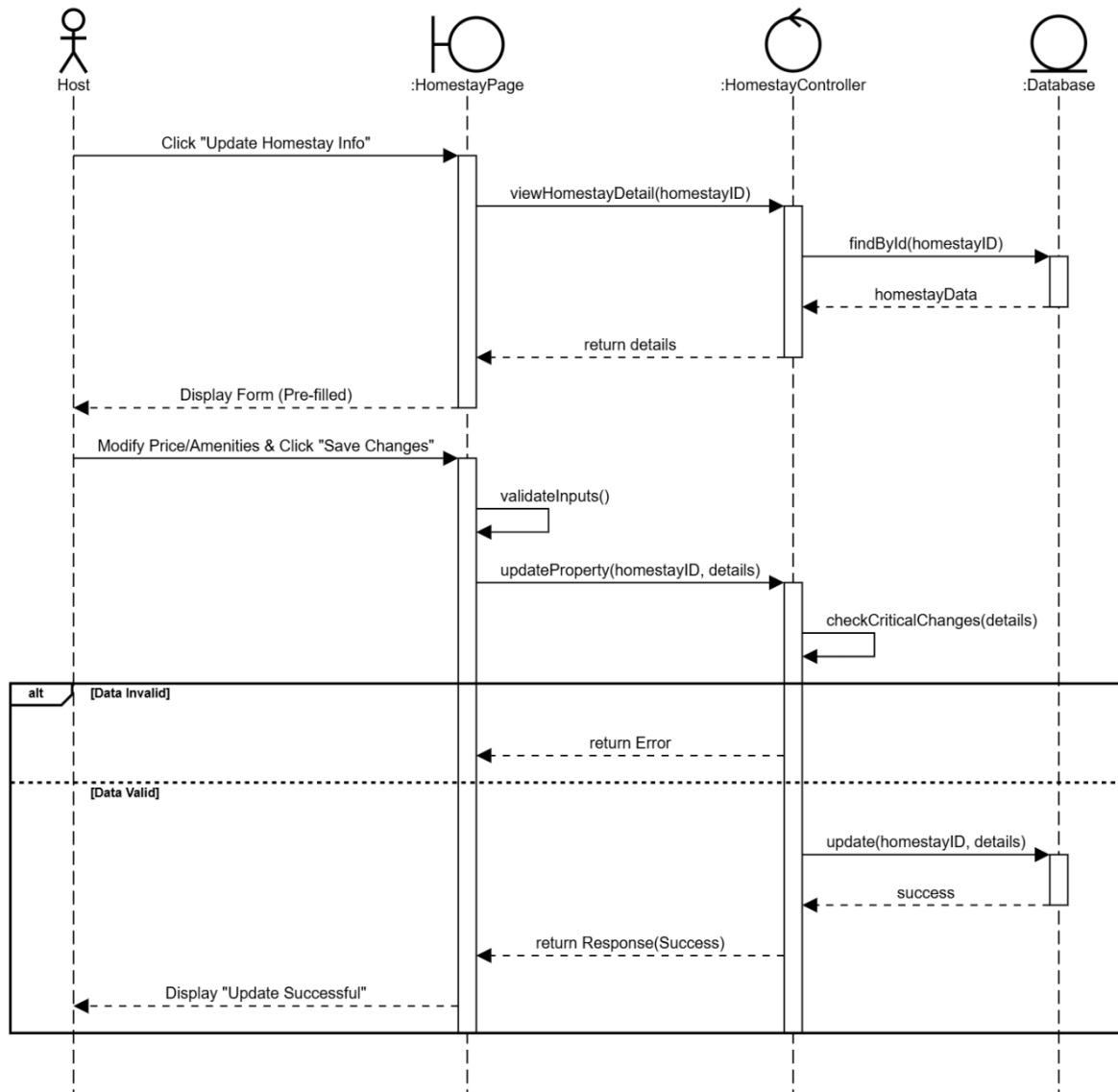
UC 2.3: Book Homestay



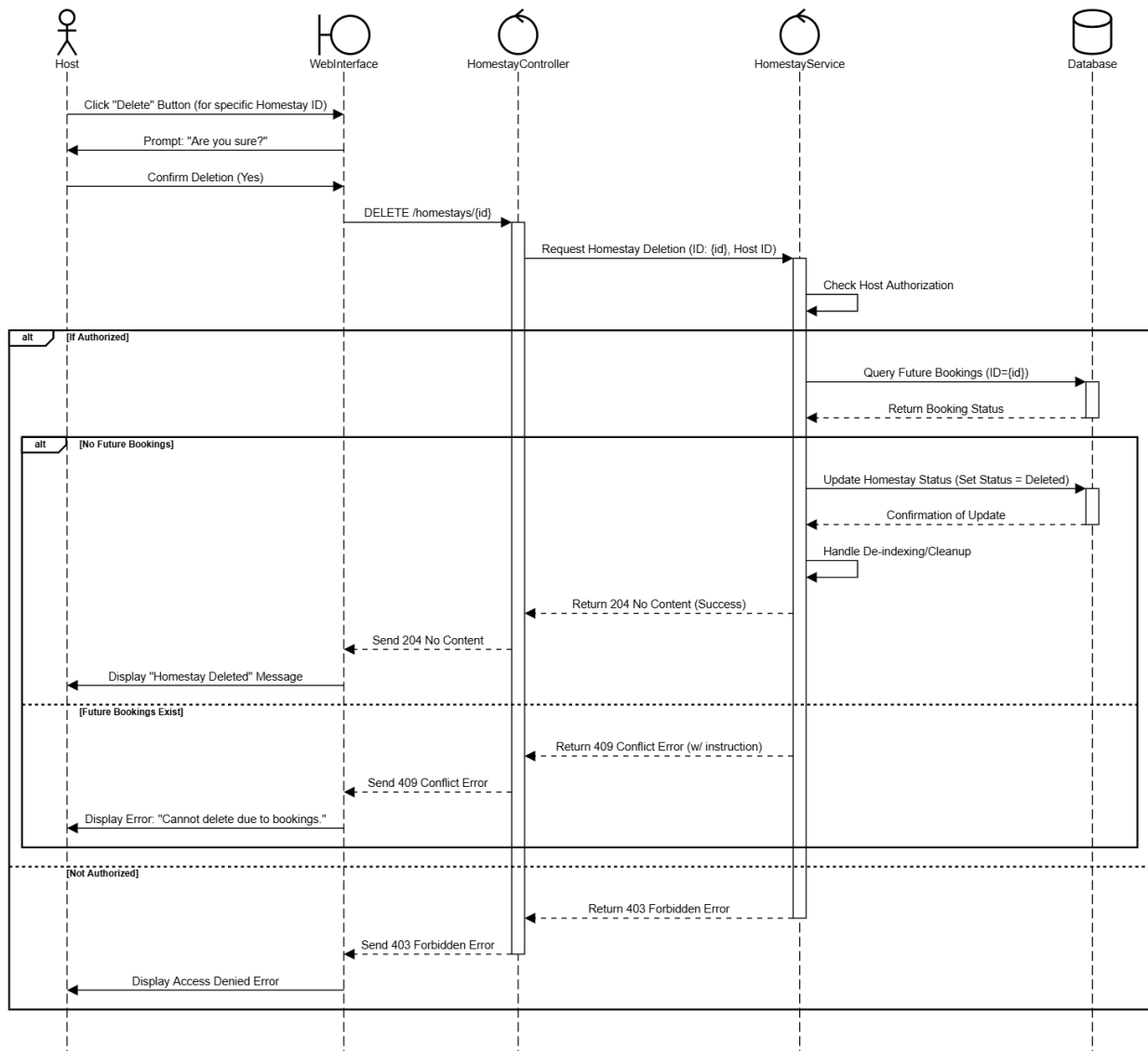
UC 3.1: Create Homestay Post



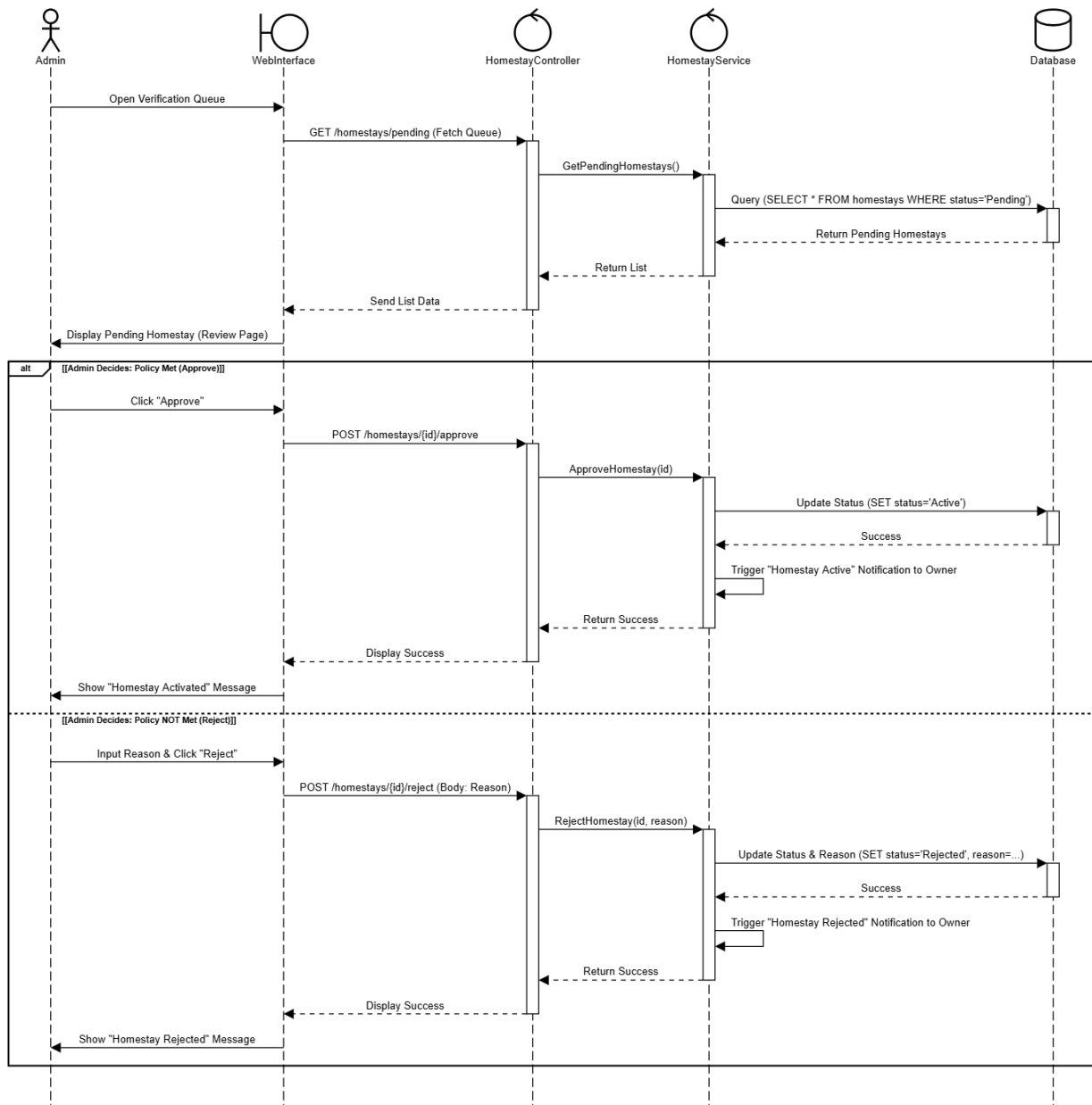
UC 3.2: Update Homestay Info



UC 3.3: Delete Homestay Post



UC 4.1: Verify Homestay Info

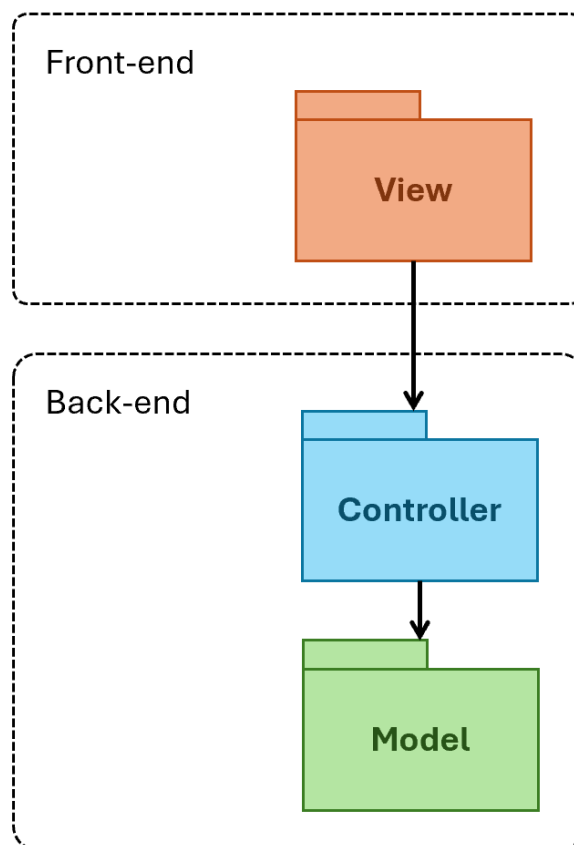


4. Design

4.1. Architecture Design

4.1.1. Package Diagram

Our project is constructed based on MVC (Model-Controller-View) Architecture. This architecture helps divide the Front-end and Back-end into separate parts. In this way, these two parts can be updated and managed independently without any conflicts.

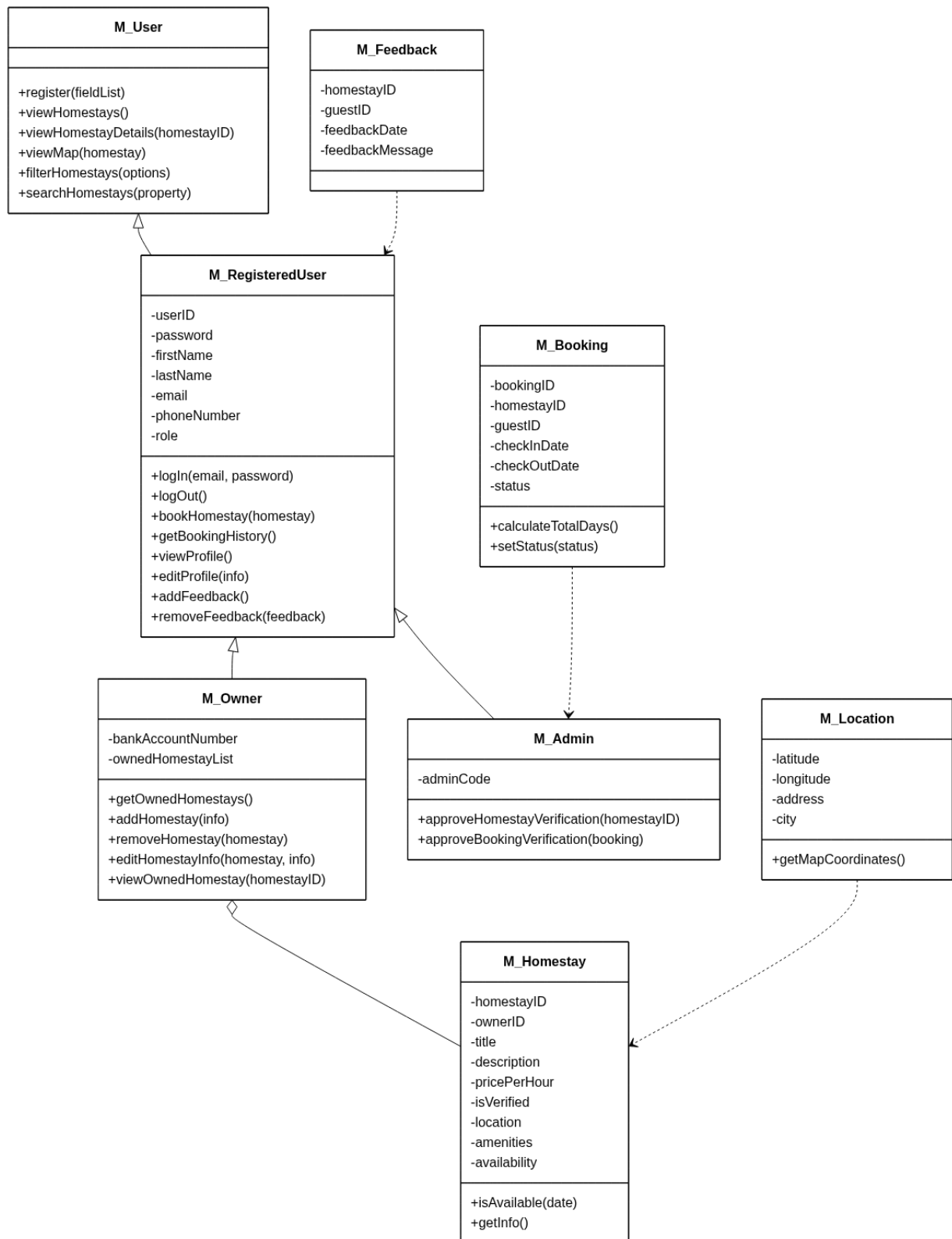


- **Model** represents the data and the business logic of the application. It handles data storage, retrieval, and processing, ensuring that the core functionality operates independently of user interfaces. For example, it manages the database, processes user input, and enforces business rules.
- **View** represents the presentation layer, responsible for displaying data to the user and capturing user input. It includes the user interface components such as web pages, forms, and visual elements, ensuring an interactive and user-friendly experience.

- **Controller** acts as the intermediary between the Model and the View. It processes user actions (e.g., button clicks, form submissions), interacts with the Model to retrieve or modify data, and updates the View accordingly. It ensures that the application responds dynamically to user input while maintaining separation of concerns.

4.2. Class Specification Design

4.2.1. Model Package



4.2.1.1 Class M_User

Description: Represents a generic user of the system, providing basic functionality for searching and viewing homestays before registration or login.

Attributes:

None

Methods:

Method	Input	Output	Access ibility	Description
register	fieldList: Object	Response	Public	Registers a new user into the system using the provided details.
view Homestays	-	Array	Public	Retrieves a list of all available homestays.
view Homestay Details	homestayID: int	Homestay	Public	Retrieves detailed information for a specific homestay.
viewMap	homestay: Homestay	-	Public	Displays the location of a specific homestay on a map interface.
filter Homestays	options: Object	Array	Public	Returns a list of homestays matching specific criteria (price, location, etc.).
search Homestays	property: String	Array	Public	Searches for homestays based on a keyword string.

4.2.1.2 Class M_RegisteredUser

Description: Represents a user who has created an account, extending the basic User capabilities with authentication and profile management.

Attributes:

Attribute	Datatype	Accessibility	Description
userID	int	Private	Unique identifier for the registered user.
password	String	Private	The user's encrypted login password.
firstName	String	Private	The user's first name.
lastName	String	Private	The user's last name.
email	String	Private	The user's email address used for login and contact.
phoneNumber	String	Private	The user's contact phone number.
role	String	Private	The role of the user (e.g., Guest, Owner, Admin).

Methods:

Method	Input	Output	Access ibility	Description
logIn	email: String, password: String	Response	Public	Authenticates the user credentials to grant access to the system.
logOut	-	-	Public	Ends the current user session.
book Homestay	homestay: Homestay	Booking	Public	Initiates a booking process for a selected homestay.
get Booking History	-	Array	Public	Retrieves a list of past and current bookings made by the user.
view Profile	-	-	Public	Displays the user's profile information.
edit Profile	info: Object	Response	Public	Updates the user's personal details.
add Feedback	-	Feedback	Public	Allows the user to create feedback for a stay.
remove Feedback	feedback: Feedback	-	Public	Deletes a previously submitted feedback entry.

4.2.1.3 Class M_Owner

Description: A specialized Registered User who owns properties and manages homestay listings.

Attributes:

Attribute	Datatype	Accessibility	Description
bankAccount Number	String	Private	The bank account details for receiving payments.
ownedHomestay List	Array	Private	A list of homestays owned by this user.

Methods:

Method	Input	Output	Access ibility	Description
getOwned Homestays	-	Array	Public	Retrieves the list of properties owned by the user.
addHomestay	info: Object	-	Public	Creates a new homestay listing.
remove Homestay	homestay: Homestay	Response	Public	Removes an existing homestay listing from the system.
editHomestay Info	homestay: Homestay, info: Object	Response	Public	Updates details of a specific homestay.
viewOwned Homestay	homestayID: int	Homestay	Public	Views the specific details of a property the user owns.

4.2.1.4 Class M_Admin

Description: A specialized Registered User responsible for system oversight and verification of content.

Attributes:

Attribute	Datatype	Accessibility	Description
adminCode	String	Private	A unique code identifying the administrator.

Methods:

Method	Input	Output	Access ibility	Description
approve Homestay Verification	homestayID: int	boolean	Public	Reviews and approves a new homestay listing for publication.
approve Booking Verification	booking: Booking	boolean	Public	Reviews and verifies a booking transaction if manual approval is needed.

4.2.1.5 Class M_Homestay

Description: Represents a property listing available for rent within the system.

Attributes:

Attribute	Datatype	Accessibility	Description
homestayID	int	Private	Unique identifier for the homestay.
ownerID	int	Private	The ID of the Owner who owns this property.
title	String	Private	The name or title of the homestay listing.
description	String	Private	Detailed description of the property features.
pricePerHour	double	Private	The cost to rent the property for one hour.
isVerified	boolean	Private	Status indicating if the Admin has verified the property.
location	Location	Private	The geographical location object of the property.
amenities	Object	Private	A collection of amenities available (WiFi, Pool, etc.).
availability	boolean	Private	Status indicating if the property is currently available for booking.

Methods:

Method	Input	Output	Accessibility	Description
isAvailable	date: Date	boolean	Public	Checks if the property is free for booking on a specific date.
getInfo	-	Object	Public	Returns a summary object of the homestay details.

4.2.1.6 Class M_Location

Description: Represents geographical data associated with a Homestay.

Attributes:

Attribute	Datatype	Accessibility	Description
latitude	double	Private	The latitude coordinate for mapping.
longitude	double	Private	The longitude coordinate for mapping.
address	String	Private	The physical street address.
city	String	Private	The city where the property is located.

Methods:

Method	Input	Output	Accessibility	Description
getMap Coordinates	-	String	Public	Returns the coordinates formatted for map API usage.

4.2.1.7 Class M_Booking

Description: Represents a reservation record connecting a Guest to a Homestay for a specific duration.

Attributes:

Attribute	Datatype	Accessibility	Description
bookingID	int	Private	Unique identifier for the booking transaction.
homestayID	int	Private	The ID of the booked homestay.
guestID	int	Private	The ID of the user who made the booking.
checkInDate	Date	Private	The date the stay begins.
checkOutDate	Date	Private	The date the stay ends.
status	String	Private	Current state of booking (e.g., Pending, Confirmed, Cancelled).

Methods:

Method	Input	Output	Accessibility	Description
calculate TotalDays	-	int	Public	Calculates duration of stay based on check-in and check-out dates.
setStatus	status: String	void	Public	Updates the status of the booking.

4.2.1.8 Class M_Feedback

Description: Represents a review or rating left by a Guest regarding a specific Homestay.

Attributes:

Attribute	Datatype	Accessibility	Description
homestayID	int	Private	The ID of the homestay being reviewed.
guestID	int	Private	The ID of the user submitting the feedback.
feedbackDate	date	Private	The date the feedback was submitted.
feedback Message	String	Private	The text content of the review.

4.2.2. View Package

V_DashboardView
-homestayList -searchBar
+displayHomestays() +navigateToHomestayDetails()

V_HomestayDetailView
-homestay
+displayHomestayDetails() +openGoogleMaps() +onBookClick()

V_SearchBar
-searchQuery -options
+displaySearchBar() +showFilterPanel() +getSearchData() +applyFilters()

V_RegistrationView
-email -password -firstName -lastName -phoneNum
+displayRegistrationForm() +showErrorMessage() +navigateToLogin() +getRegistrationData()

V_LoginView
-email -password
+displayLoginForm() +showErrorMessage() +redirectToDashboard() -validateInputs()

V_BookingView
-checkInDate -checkOutDate
+displayBookingForm() +showSuccessMessage() +showErrorMessage() +getBookingData()

V_HomestayManagementView
-ownerHomestays
+displayHomestays() +createHomestay() +updateHomestay() +deleteHomestay() -validateInputs()

V_HomestayFormView
-homestay
+displayCreateForm() +displayUpdateForm()

V_AdminVerificationView
-pendingHomestays
+displayPendingHomestays() +displayHomestayDetails() +approveHomestay() +rejectHomestay()

4.2.2.1. Class V_DashboardView

Description: Main interface displaying list of homestays

Attributes:

Attribute	Datatype	Access ibility	Description
homestay List	List<Homestay>	Private	Collection of homestays to display
searchBar	V_SearchBar	Private	An instance of the search bar class

Methods:

Method	Input	Output	Access ibility	Description
display Homestays	homestays: Array	void	Public	Renders homestay list
navigateTo HomestayDetails	homestayId: int	Response	Public	Redirects to detail view

4.2.2.2. Class V_HomestayDetailView

Description: Displays complete information about a specific homestay

Attributes:

Attribute	Datatype	Accessibility	Description
homestay	Homestay	Private	Current homestay to be displayed

Methods:

Method	Input	Output	Access ibility	Description
Display Homestay Details	homestay: Homestay	void	Public	Shows all homestay information (e.g photos, amenities, price, description
Open Google Maps	-	void	Public	Launches Google Maps
onBook Click	-	void	Public	Open booking view

4.2.2.3. Class V_SearchBar

Description: Search interface component

Attributes:

Attribute	Datatype	Accessibility	Description
searchQuery	String	Private	Current search text
options	Object	Private	Current filter settings

Methods:

Method	Input	Output	Accessibility	Description
display SearchBar	-	void	Public	Renders search input field
showFilter Panel	-	Options	Public	Display filter options
getSearchDat	-	Objects	Public	Returns query string and selected filter options
applyFilters	-	Array	Public	Triggers filter application

4.2.2.4. Class V_RegistrationView

Description: User registration interface

Attributes:

Attribute	Datatype	Accessibility	Description
email	String	Private	Email input
password	String	Private	Password input
firstName	String	Private	First Name input
lastName	String	Private	Last Name input
phoneNum	String	Private	Phone number input

Methods:

Method	Input	Output	Accessibility	Description
display Registration Form	-	void	Public	Shows registration form
showError Message	message: String	-	Public	Display error message
navigate ToLogin	-	-	Public	Refirects to login page
getRegistrationData	-	Object	Public	Return data object for controller to process

4.2.2.5. Class V_LoginView

Description: User authentication interface

Attributes:

Attribute	Datatype	Accessibility	Description
email	String	Private	Email input field
password	String	Private	Password input field

Methods:

Method	Input	Output	Accessibility	Description
displayLogin Form	-	void	Public	Show login form
showError Message	message: String	void	Public	Display error message
redirectTo Dashboard	-	void	Public	Redirects to dashboard
validateInputs	-	boolean	Private	Check if fields are empty or if email format is valid

4.2.2.6. Class V_BookingView

Description: Homestay booking interface

Attributes:

Attribute	Datatype	Accessibility	Description
checkInDate	Datetime	Private	Check-in time selection
checkOutDate	Datetime	Private	Check-out time selection

Methods:

Method	Input	Output	Access ibility	Description
displayBooking Form	homestay: Homestay	void	Public	Show form
showSuccess Message	message: String	void	Public	Display booking confirmation message
showError Message	message: String	void	Public	Display error message
getBookingData	-	void	Public	Return data object (dates & details) for controller

4.2.2.7. Class V_HomestayManagementView

Description: Owner's homestay management interface

Attributes:

Attribute	Datatype	Accessibility	Description
ownerHomestays	Array	Private	Owner's list of homestays

Methods:

Method	Input	Output	Access ibility	Description
display Homestays	homestays: Array	void	Public	Shows owner's list of homestays
create Homestay	-	void	Public	Opens create form
update Homestay	details: Homestay	void	Public	Opens pre-filled update form
delete Homestay	homestayId: int	void	Public	Display delete confirm dialog
validateInputs	formData: Object	boolean	Private	Checks if required fields (price, address) are filled and valid.

4.2.2.8. Class V_HomestayFormView

Description: Form for creating or updating homestay profiles

Attributes:

Attribute	Datatype	Accessibility	Description
homestay	Homestay	Private	Homestay data (null for create, populated for update)

Methods:

Method	Input	Output	Accessibility	Description
display CreateForm	-	Form	Public	Shows empty form for new homestay
display UpdateForm	homestay: Homestay	Form	Public	Shows pre-filled form

4.2.2.9. Class V_AdminVerificationView

Description: Admin interface for verifying homestay details

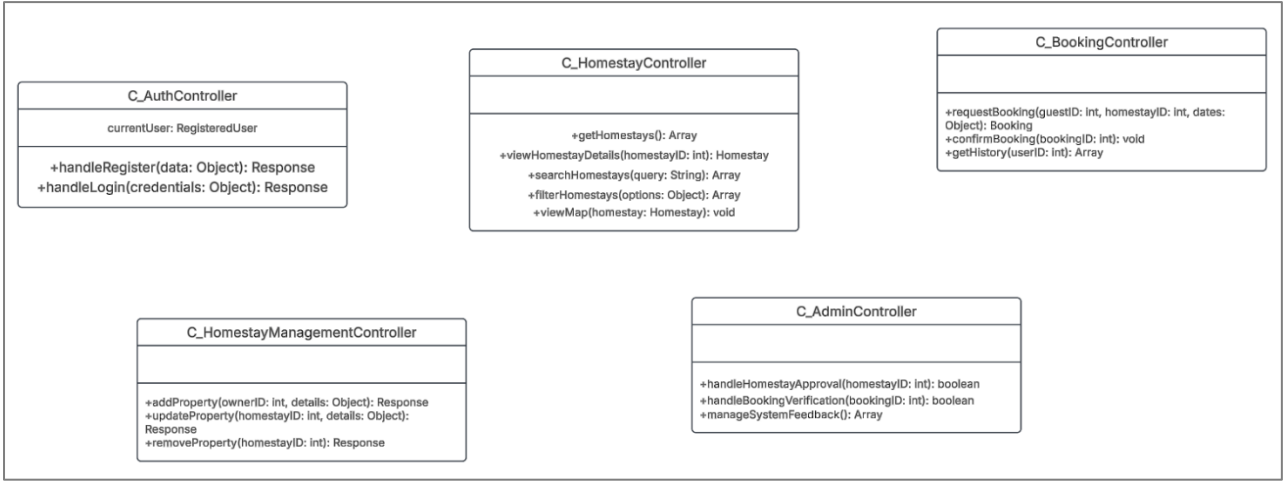
Attributes:

Attribute	Datatype	Accessibility	Description
pendingHomestays	Array	Private	Homestays awaiting approval

Methods:

Method	Input	Output	Access ibility	Description
displayPending Homestays	homestays: Array		Public	Shows pending list of homestays
display Homestay Details	homestay: Homestay	Details	Public	Shows detailed view of homestay
approve Homestay	HomestayId: int	Success/Fail	Public	Processes approval
reject Homestay	homestayId: int, reason: String	Success/Fail	Public	Processes rejection

4.2.3. Control Package



4.2.3.1. Class C_AuthController

Attributes:

Attribute	Datatype	Accessibility	Description
currentUser	RegisteredUser	Private	Stores the current authenticated user in the active session/context.

Methods:

Method	Input	Output	Access ibility	Description
handle Register	data: Object	Response	Public	Handles user registration based on the provided user information
handle Login	credentials: Object	Response	Public	Handles user login using the provided credentials (e.g., email/phone and password).

4.2.3.2. Class C_BookingController

Attributes:

None

Methods:

Method	Input	Output	Access ibility	Description
request Booking	guestID: int, homestayID: int, dates: Object	Response	Public	Creates/submits a booking request for a guest at a specific homestay within the given dates.
confirm Booking	bookingID: int	void	Public	Confirm the booking identified by BookingID
getHistory	userID: int	Array	Public	Retrieves the booking history of a user (list of booking)

4.2.3.3. Class C_HomestayManagementController

Attributes:

None

Methods:

Method	Input	Output	Access ibility	Description
addProperty	ownerID: int, details:Object	Response	Public	Adds a new homestay/property for the specified owner using the provided details
update Property	homestayID: int, details:Object	Response	Public	Updates an existing homestay/property identified by homestayID with the given details
remove Property	homestayID: int	Response	Public	Removes a homestay identified by homestayID from the system (or archives it, depending on design).

4.2.3.3. Class C_HomestayManagementController

Attributes:

None

Methods:

Method	Input	Output	Access ibility	Description
request Booking	guestID: int, homestayID: int, dates: Object	Response	Public	Creates/submits a booking request for a guest at a specific homestay within the given dates.
confirm Booking	bookingID: int	void	Public	Confirm the booking identified by BookingID
getHistory	userID: int	Array	Public	Retrieves the booking history of a user (list of booking)

4.2.3.4. Class C_AdminController

Attributes:

None

Methods:

Method	Input	Output	Access ibility	Description
handleHomestay Approval	homestayID: int	boolean	Public	Allows an admin to verify new homestays
handleBooking Verification	bookingID: int	boolean	Public	Allows an admin to verify bookings
manageSystem Feedback		Array	Public	Managing users' feedbacks on the system

4.2.3.5. Class C_HomestayController

Attributes:

None

Methods:

Method	Input	Output	Access ibility	Description
get Homestays	-	Array	Public	Retrieves a list of available homestays.
View Homestay Details	homestayID: int	Homestay	Public	Retrieves detailed information of a homestay identified by homestayID
search Homestays	query: String	Array	Public	Searches homestays based on the given query keyword(s) and returns matching results.
filter Homestays	options: Object	Array	Public	Filters homestays using the provided options.
viewMap	homestay: Homestay	void	Public	Displays the homestay location on a map for the given homestay.

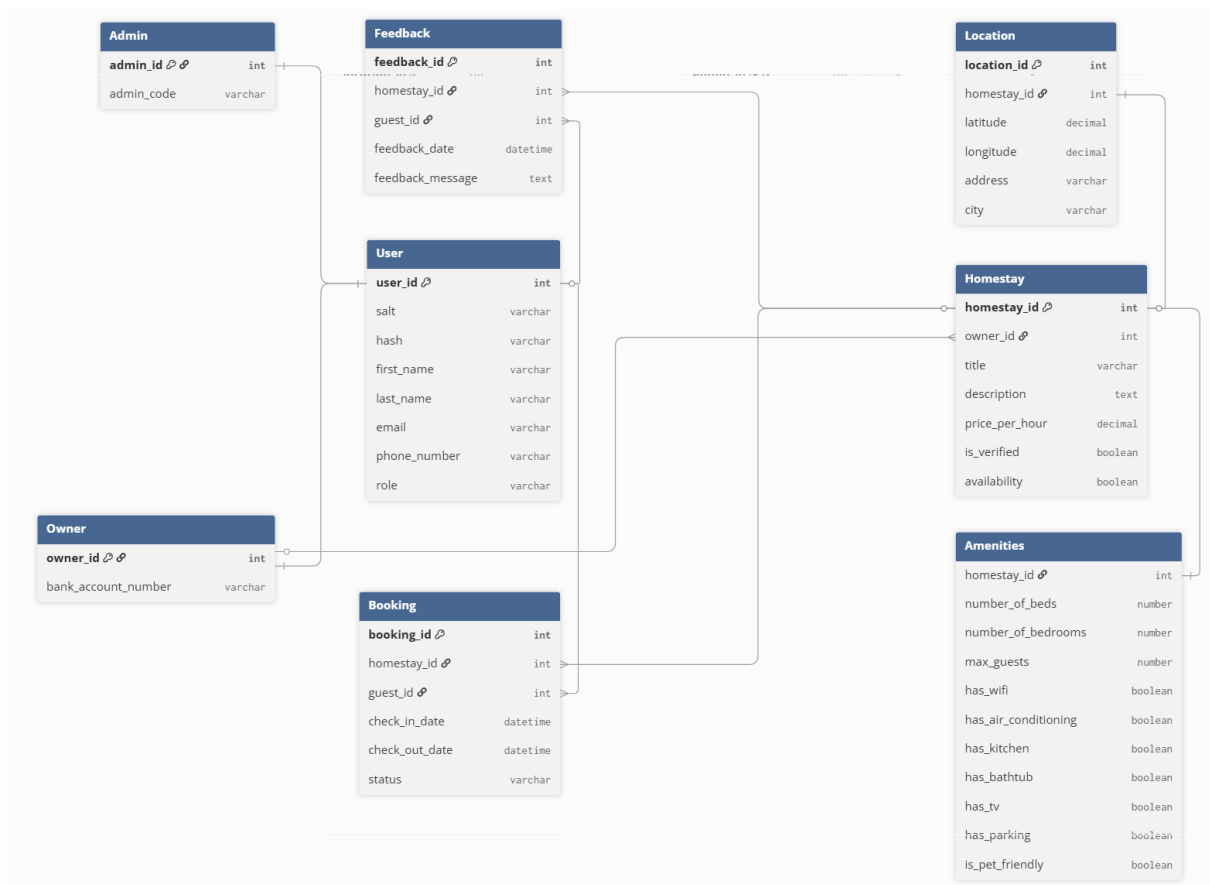
4.3. Data Storage Design

4.3.1. Overview

The database design outlines the data architecture for a Homestay Management System, meant to facilitate interactions between property owners, guests, and administrative staff. The system centralizes the management of property listings, secure user authentication, reservation tracking, and service quality feedback.

The database is organized into four logical clusters:

- **Identity & Access (User, Admin, Owner):** Uses a specialized "User" table as a base, with Admin and Owner tables extending it to store role-specific data.
- **Property Catalog (Homestay, Amenities, Location):** The Homestay entity is supported by a one-to-one relationship with Amenities for capturing specific features, and Location for storing spatial and address data.
- **Booking:** Connects users and owners to homestays, recording essential temporal data like `check_in_date` and `check_out_date`.
- **Feedback:** Links guests back to the homestays they visited to store ratings and qualitative reviews.



Index	Table Name	Description
1	User	The central identity table storing core credentials and profile information for registered individuals using the platform.
2	Admin	An extension of the User table that stores specialized access codes for platform moderators and staff.
3	Owner	An extension of the User table containing financial details for owners.
4	Homestay	The primary entity for property listings, tracking title, description, hourly pricing, and verification status.
5	Location	Stores the physical address and precise geographic coordinates (latitude/longitude) for each homestay.
6	Amenities	A detailed checklist of property features offered by homestays.
7	Booking	Manages the transactional relationship between guests and homestays.
8	Feedback	Captures post-stay reviews and messages from guests.

4.3.2. Detailed Data Storage Design

User Table

Index	Field name	Data-type	Null-able	Const-raint	Description
1	user_id	INT	No	Primary Key	Unique identifier for each user (Auto-increment).
2	hash	VARCHAR (255)	No	-	The cryptographically hashed password.
3	salt	VARCHAR (128)	No	-	A unique, random string added to the password before hashing.
4	first_name	VARCHAR (50)	No	-	The user's given name.
5	last_name	VARCHAR (50)	No	-	The user's family name.
6	email	VARCHAR (255)	No	Unique	User's email address
7	phone_number	VARCHAR (20)	Yes	-	Contact number including international country codes and formatting.
8	role	VARCHAR (5)	No	-	Defines permissions (“Guest”, “Owner”, “Admin”)

Admin Table

Index	Field name	Data-type	Null-able	Constraint	Description
1	admin_id	INT	No	Primary Key, Foreign Key	References User(user_id) to inherit core profile data.
2	admin_code	VARCHAR (20)	No	Unique	A unique internal identifier used for administrative verification.

Owner Table

Index	Field name	Data-type	Null-able	Constraint	Description
1	owner_id	INT	No	Primary Key, Foreign Key	References User(user_id). Ensures the owner exists as a registered user.
2	bank_account_number	VARCHAR (34)	No	-	Stores the account for payouts.

Homestay Table

Index	Field name	Data-type	Null-able	Const-raint	Description
1	homestay_id	INT	No	Primary Key	Unique identifier for the homestay listing (Auto-increment).
2	owner_id	INT	No	Foreign Key	References Owner(owner_id) to identify the legal host.
3	title	VARCHAR (150)	No	-	The display name of the property.
4	description	TEXT	No	-	Detailed marketing text describing the property and homestay rules.
5	price_per_hour	DECIMAL (10,2)	No	-	The renting price of the homestay per hour.
6	is_verified	BOOLEAN	No	Default: False	Indicates if the admin has audited the property for authenticity.
7	availability	BOOLEAN	No	Default: True	Indicates if the homestay is available for renting or not.

Booking Table

Index	Field name	Data-type	Null-able	Constraint	Description
1	booking_id	INT	No	Primary Key	Unique identifier for the reservation (Auto-increment).
2	homestay_id	INT	No	Foreign Key	References Homestay(homestay_id) to identify the reserved property.
3	guest_id	INT	No	Foreign Key	References User(user_id) to identify the guest making the booking.
4	check_in_date	DATETIME	No	-	The scheduled start date and time for the stay.
5	check_out_date	DATETIME	No	-	The scheduled end date and time for the stay.
6	status	VARCHAR (20)	No	-	The current state ("Pending", "Confirmed", "Cancelled", "Completed").

Amenities Table

Index	Field name	Data-type	Null-able	Const-raint	Description
1	homestay_id	INT	No	Primary Key, Foreign Key	References Homestay(homestay_id). Acts as the identifier for the amenities record.
2	number_of_beds	INT	No	-	The total count of beds available for guests.
3	number_of_bedrooms	INT	No	-	The total count of separate sleeping rooms.
4	max_guests	INT	No	-	The maximum occupancy allowed for the property.
5	has_wifi	BOOLEAN	No	Default: False	Flag indicating if wireless internet is provided.
6	has_air_conditioning	BOOLEAN	No	Default: False	Flag indicating if the property has AC.
7	has_kitchen	BOOLEAN	No	Default: False	Flag indicating if cooking facilities are available.
8	has_bathtub	BOOLEAN	No	Default: False	Flag indicating if there is a bathtub in the bathroom.
9	has_tv	BOOLEAN	No	Default: False	Flag indicating if a television is provided.
10	has_parking	BOOLEAN	No	Default: False	Flag indicating if on-site parking is available.
11	is_pet_friendly	BOOLEAN	No	Default: False	Flag indicating if guests are allowed to bring pets.

Feedback Table

Index	Field name	Data-type	Null-able	Constraint	Description
1	feedback_id	INT	No	Primary Key	Unique identifier for each review (Auto-increment).
2	homestay_id	INT	No	Foreign Key	References Homestay(homestay_id) to identify the property being reviewed.
3	guest_id	INT	No	Foreign Key	References User(user_id) to identify the guest who wrote the review.
4	feedback_date	DATETIME	No	-	The timestamp of when the feedback was submitted.
5	feedback_message	TEXT	Yes	-	The qualitative review text.

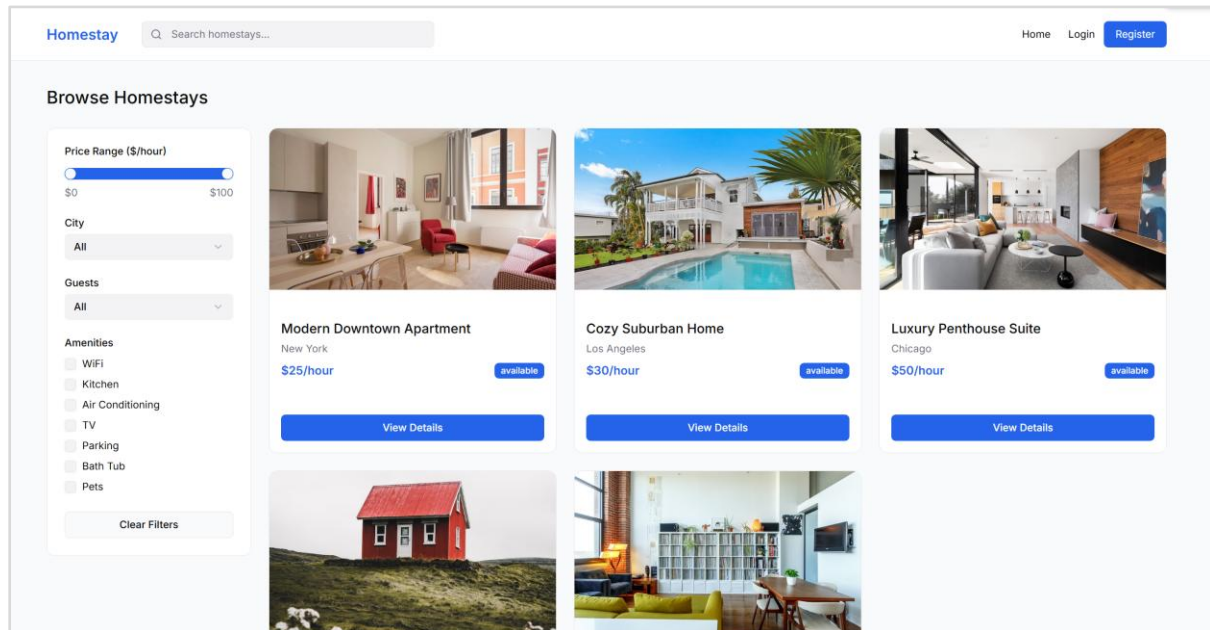
Location Table

Index	Field name	Data-type	Null-able	Const-raint	Description
1	location_id	INT	No	Primary Key	Unique identifier for the location record.
2	homestay_id	INT	No	Foreign Key	References Homestay(homestay_id) to link the address to a homestay.
3	latitude	DECIMAL (10, 8)	No	-	Geographic coordinate.
4	longitude	DECIMAL (11, 8)	No	-	Geographic coordinate.
5	address	VARCHAR (255)	No	-	The full street address of the property.
6	city	VARCHAR (100)	No	-	The city or municipality where the homestay is located.

4.4. User Interface Design

4.4.1. Interface Prototype

4.4.1.1. Homepage



4.4.1.2. Register Form

Register

Create your account to start booking homestays

First Name

Last Name

Email

Phone

Password

Confirm Password

Register

[Already have an account? Login](#)

4.4.1.3. Login Form

Login

Enter your credentials to access your account

Email

you@example.com

Password

.....

Login

Don't have an account? [Register](#)

4.4.1.4. Logout Form

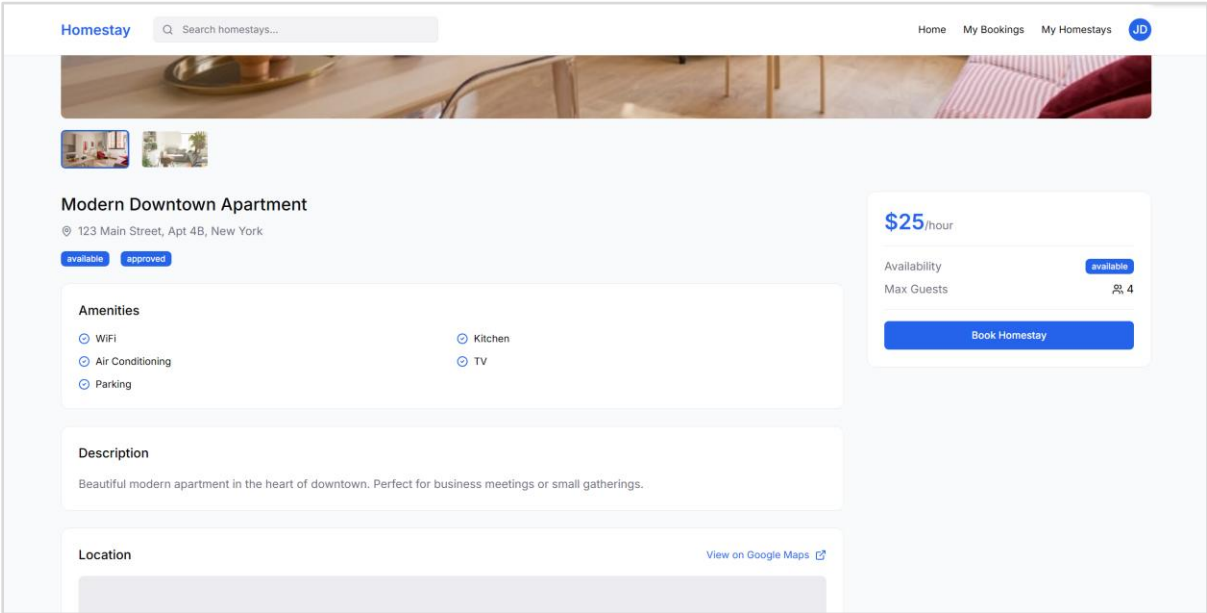
HomeMy BookingsMy Homestays

JD

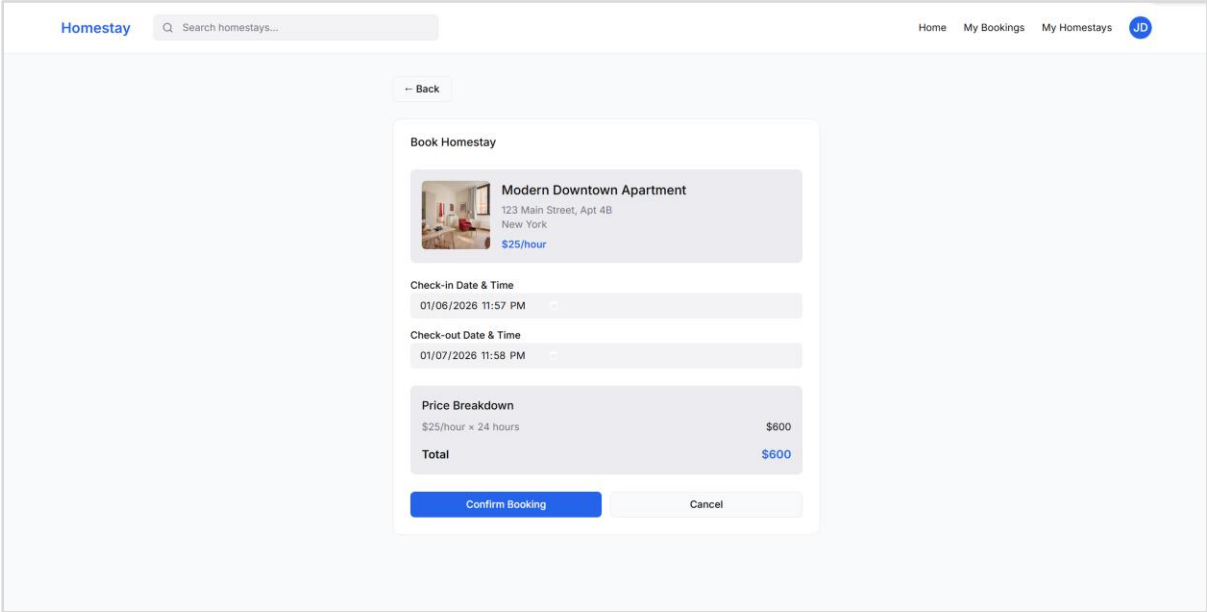
John Doe
owner@example.com

[→ Logout]

4.4.1.5. Homestay Detail View



4.4.1.6. Booking View



4.4.1.7. My Booking View

Homestay

Search homestays...

HomeMy BookingsMy HomestaysJD

My Bookings

Your Booking History

Homestay	Check-in	Check-out	Total Price	Status	Booked On	Actions
Modern Downtown Apartment	Feb 01, 2026 17:00	Feb 01, 2026 23:00	\$150	pending	Jan 15, 2026	Cancel
Cozy Suburban Home	Feb 05, 2026 21:00	Feb 06, 2026 03:00	\$180	approved	Jan 16, 2026	View Details

4.4.1.8. Homestay Form View

Homestay

Search homestays...

HomeMy BookingsMy HomestaysJD

Create Homestay

Title

City

Address

Price per Hour (\$)

Max Guests

Amenities

☐ WiFi

☐ TV

☐ Pet

☐ Kitchen

☐ Parking

☐ Air Conditioning

☐ Bath Tub

Description

Images

Choose Files No file chosen

Upload images of your homestay

Submit Homestay

Cancel

Your homestay will be pending admin approval after submission.

4.4.1.9. Admin Panel

Homestay

Q Search homestays ...

HomeMy Booking

Admin - Verify Homestays

Pending Verification (1)

Homestay	Owner	City	Price/Hour	Date Submitted
Beach House Retreat	Owner 2	Miami	\$40	Jan 04, 2026

All Homestays

Homestay	Owner	City	Status	Availability
Modern Downtown Apartment	Owner 2	New York	approved	available
Cozy Suburban Home	Owner 2	Los Angeles	approved	available
Luxury Penthouse Suite	Owner 2	Chicago	approved	available
Beach House Retreat	Owner 2	Miami	pending	available
Mountain Cabin Escape	Owner 2	Denver	approved	unavailable
Urban Loft Studio	Owner 2	San Francisco	approved	available

Review Homestay

Review the homestay details and approve or reject the listing.



Beach House Retreat

321 Ocean Drive
Miami

Price per Hour

\$40

Max Guests

10

Status

pending

Amenities

WiFiKitchenParkingBath TubPets

Description

Amazing beach house with direct ocean access. Perfect for summer parties and relaxation.

Submitted On

January 04, 2026

Approve

Reject

4.4.2. Interface Specification

4.4.2.1. Homepage

Interface	Homepage
Content	Interface for guest users. Select Login, Register or View and Search Homestays
Condition	None
Complexity	Simple

4.4.2.2. Register Form

Interface	Homepage
Content	Interface for registering new accounts
Condition	User select register on Homepage
Complexity	Simple

4.4.2.3. Login Form

Interface	Homepage
Content	Interface for logging in to active accounts
Condition	a. Users select Login on Homepage b. Redirect after registering a new account
Complexity	Simple

4.4.2.4. Logout Form

Interface	Homepage
Content	Interface for logging out from active accounts
Condition	Logged in user select Logout from Homepage.
Complexity	Simple

4.4.2.5. Homestay Detail View

Interface	Homepage
Content	Interface for viewing details and book homestay.
Condition	User select View Details of a homestay on Homepage or Search Result
Complexity	Simple

4.4.2.6. Booking View

Interface	Homepage
Content	Interface for booking homestay
Condition	User select Book Homestay on Homestay Detail View
Complexity	Simple

4.4.2.7. My Booking View

Interface	Homepage
Content	Interface for booking history
Condition	User select My Bookings after logging in
Complexity	Simple

4.4.2.8. Homestay Create Form View

Interface	Homepage
Content	Interface for creating new Homestay profile
Condition	Owners select Create Homestay on My Homestay View
Complexity	Simple

4.4.2.9. Admin Panel

Interface	Homepage
Content	Interface for verifying submitted homestay profiles
Condition	Admin select Verify Homestays
Complexity	Simple

5. Conclusion and Future Work

5.1. Conclusion

Following a comprehensive phase of requirements analysis and system design, this project presents a proposed architecture and specification for the Homestay Booking Management System. The platform is designed to serve as a transparent, highly automated ecosystem that supports the complete guest journey, from initial property discovery to post-stay coordination, while improving operational efficiency for hosts.

The proposed solution consolidates essential modules, including property profile syndication, secure booking workflows, identity verification requirements, and housekeeping task orchestration within a centralized data model. A key design principle is an event-triggered data-capture policy, where critical user and system interactions are recorded as events to strengthen auditability, enable dispute handling, and support privacy-aware governance and stakeholder trust.

From a technical perspective, the design specifies a modern web-based architecture. The frontend layer is intended to leverage HTML, CSS, JavaScript, and React to provide a responsive and consistent interface across devices. The backend layer is defined as an API-driven service (e.g., Node.js/Express or Java Spring Boot) responsible for authentication and authorization, booking logic and concurrency control, and secure integration with external services such as payment gateways via encrypted communication. In addition, cross-browser compatibility requirements are identified for widely used browsers including Google Chrome, Mozilla Firefox, and Safari. The interface design also includes key user flows and screen-level specifications for listing management, search and filtering, booking and payment steps, and customer support interactions.

To support iterative validation and refinement of requirements, an Agile-based delivery approach is recommended, enabling continuous stakeholder feedback throughout future implementation phases. Furthermore, the design highlights non-functional requirements such as reliability, maintainability, scalability, and security, and recommends operational practices (e.g., observability, versioned releases, and monitoring) to be implemented when the system is developed and deployed.

Several technical and regulatory considerations were identified during the analysis. Integration with legacy property databases is recognized as a major risk area, and the design recommends synchronization strategies and data consistency controls to reduce overbooking scenarios. In addition, security requirements are specified to protect personal and financial data, including strong authentication, least-privilege access control, and encryption in transit and at rest to mitigate emerging cyber threats.

In summary, this project delivers a scalable and secure system design blueprint for a Homestay Booking Management System. By addressing key architectural decisions, data modeling, and risk mitigation strategies at the design stage, the proposed solution provides a solid foundation for future implementation and evaluation.

5.2. Future Work

While the system successfully provides a reliable and convenient user experience, the following enhancements are proposed to further optimize the platform:

- **Personalized Recommender System:** Implementing a data-driven engine that utilizes machine learning to suggest properties based on individual user preferences and historical behavior.
- **Collaborative Group Bookings:** Introducing a feature that allows groups to secure multiple properties in the same vicinity while facilitating individual payment split options.
- **AI-Powered Smart Concierge:** Integrating a 24/7 AI chatbot to handle routine guest inquiries, provide local recommendations, and offer instant troubleshooting.
- **Dynamic Pricing Analytics:** Developing an automated tool for hosts that adjusts rental rates in real-time based on seasonal demand, local events, and market trends.
- **Immersive Virtual Tours:** Incorporating 360-degree VR walkthroughs to allow prospective guests to explore property interiors remotely, increasing booking confidence and transparency.

References

1. *Systems Analysis and Design, 8th Edition (Alan Dennis, Barbara Wixom, Roberta M. Roth).*
2. *Object-Oriented Analysis and Design Understanding System Development with UML 2.0 by Mike O'Docherty.*
3. *Decree-No-13-2023-ND-CP April-17-2023 on protection of personal data.*